

Supplementary material

Supplement to “In a single pass, most local 4-bit executors transcribe a detailed plan; two of thirteen bind it”. The title’s terms recur throughout this document, so we define them here. A single pass is one generation attempt: the model emits a script, and no feedback ever returns to it. A model transcribes a plan when its emitted script reproduces the plan’s commands, and binds the plan when the plan’s content, rather than the model’s own defaults, determines what the script does.

Contents:

- **Supplementary Methods** (Tables S1–S9): study-design ledger; plan conditions and prompts; data-governance audit; hardware sizing and residency; inference settings; execution environment; scoring conventions; error injection; benchmark-2 protocol; the fabrication study; platforms and reproducibility metadata; the cost model.
- **Supplementary Results** (Tables S10–S18): generation outcomes; per-model repeated sampling; conventional variant-calling metrics; the emitted-script constraint audit; the reasoning arms; wall time by platform; benchmark-2 per-sample counts and the defect ledger; the seed extension at the discriminating conditions; the perturbed-plan pass; the bounded repair arm, in which a failed script could be resubmitted up to three attempts with the failing script and its error message shown to the model between attempts.
- **Supplementary Discussion**: the workflow-class taxonomy (Table S19); production concerns for a shared deployment (Table S20); the agent-harness survey.
- **Verbatim prompts and plan files.**
- **Supplementary Figures S1–S3.**

Supplementary Methods

Study design: the cell ledger and the four denominators

Table S1. Cell-count and wall-clock ledger for benchmark 1, one row per released log file; Track A denotes runs whose executor prompt carried the named plan, and Track B, defined below, denotes runs with no plan at all. Cells, generation hours and the non-ok count are now all emitted by the same pass over the released logs. The first version of this table hand-entered the non-ok column, which is why the reasoning-effort row read 0 where its log carries 2 and the total read 50 where it is 52. The frontier reference block ran on the Anthropic API and consumed no local GPU time; every other block ran on the Jetson AGX Orin.

Block	Log file	Cells	Generation (h)	Non-ok generations
Frontier reference gradient (API)	matrix_jetson.jsonl	81	1.55	0
Plan gradient, reasoning off	matrix_jetson_thinkoff.jsonl	324	7.22	1
Plan gradient, reasoning on	matrix_jetson_thinkon.jsonl	270	28.54	18
Repeated sampling at v2	matrix_jetson_n10_v2.jsonl	84	1.61	1
Track A (7 added seeds)				
qwen3.8:27b gradient, reasoning off	matrix_jetson_qwen38_off.jsonl	27	0.64	0
qwen3.8:27b gradient, reasoning on	matrix_jetson_qwen38_on.jsonl	27	17.40	27
Budget ablation at 32,768 tokens	ablation_32768.jsonl	3	4.57	3
Budget ablation at 65,536 tokens	ablation_65536.jsonl	3	7.87	0
Reasoning-effort sweep, gpt-oss:20b	effort_gptoss.jsonl	20	1.20	2

Block	Log file	Cells	Generation (h)	Non-ok generations
Reasoning-effort sweep at 128k context	<code>effort_gptoss_high128k.jsonl</code>	9	4.67	0
Total		848	75.27	52

The reasoning-effort row is 20 cells, not the 27 its design implies: the high-effort arm at a 32,768-token budget terminated after 2 of its 9 cells, both `truncated_in_thinking` at plan v1. That arm was then re-run at the model’s native 131,072-token context as a separate block, and we state the substitution wherever the effort result is reported. Two log files post-date this ledger and are not in it. `matrix_jetson_seedext.jsonl` (273 cells) is the seed extension, a re-run of the three discriminating conditions v1g, v1.25 and v1.5 across added sampler seeds. `matrix_jetson_perturbed.jsonl` (39 cells) is the perturbed-plan pass, in which each executor received a deliberately altered plan to test whether the plan is bound rather than merely transcribed. Both are reported in Tables S16 and S17. **Table S2.** Denominator reconciliation: every count is a `find` or a filter over the released tree, not a transcription.

Quantity	n	What it is
Cells run and logged	848	every row of the ten log files in Table S1
Cells archived under <code>revision/runs/</code>	813	all blocks except the budget ablation (6 cells, <code>revision/runs_ablation/</code>) and the reasoning-effort sweep (29 cells, <code>revision/runs_effort/</code>)
Archived cells carrying a <code>run.sh</code>	1,039	the emitted-script constraint audit corpus (Supplementary Results, <i>What the emitted scripts actually did</i>)
Archived cells carrying no <code>run.sh</code>	47	exactly the 47 non-ok generations in those blocks: 1 + 18 + 1 + 27 from the four gradient and sampling passes; the other 5 non-ok cells are in the ablation and effort blocks
Reasoning-off scripts	787	the transcription-index corpus: 706 local plus 81 API, including the 273 seed-extension scripts

Plan conditions, the planner meta-prompts, and the executor prompt

Eight conditions span the plan gradient. They run from no plan at all — Track B, in which the executor receives only the problem statement and the tool inventory — to a detailed plan wrapped in defensive prose (v2_defensive):

Condition	Size	Summary	Hypothesis tested
Track B	1.2 KB	Problem statement and tool inventory only	How much does a model do unaided?
v0.5	1.4 KB	Track B plus one line giving the tool order	Does sequencing alone help?
v1 (lean)	3.1 KB	Numbered bullets naming tools and key flags; no code	Reference lean plan
v1.25	3.1 KB	v1 plus one literal <code>lofreq call-parallel</code> command line	Does one full command bridge v1 to v2?
v1.5	1.3 KB	v2 with all prose stripped; code fences only	Does the prose change outcomes, or only the code?

Condition	Size	Summary	Hypothesis tested
v1g	4.2 KB	v1 plus a <code>lofreq</code> invocation extracted mechanically from the Galaxy IUC tool registry	Can a tool registry substitute for a plan author?
v2 (detailed)	4.6 KB	Exact command line and a Gotchas block per step	Reference detailed plan
v2_defensive	6.5 KB	v2 plus <code>try()</code> helper, per-step validation, retry-once, failure log	Does error-handling prose change runtime behaviour?

Three planner meta-prompts produced these files, and because the plan gradient is the study’s independent variable, how it was elicited is not an appendix detail. All three prompts are reproduced verbatim in this Supplement and are released as `plan/PLANNER_PROMPT.md`, `plan/PLANNER_PROMPT_v2.md` and `plan/PLANNER_PROMPT_v2_defensive.md`.

`PLANNER_PROMPT.md` produced v1, and editing v1 produced v1.25 and v1.5. The prompt frames the audience as “a junior bioinformatician” and “a competent shell programmer who knows generic bioinformatics tools but not the specific best-practice flags”. It forbids bash code outright (“No code blocks”), caps the plan at ten steps and 400 words, and requires flag values to be named but not shown as invocations; it requests no validation or retry prose. `PLANNER_PROMPT_v2.md` produced v2 and frames the implementer as someone who “has NEVER used bwa, samtools, lofreq, bcftools, or tabix”. It requires “the exact command line (or pipeline)” as “a fenced one-liner of code” for every step, together with “every flag, with its exact value”. It further requires an explicit idempotency guard per step and a per-step gotchas block. It states the audience constraint that “the implementer will copy-paste your invocations into a shell script” and sets no limit on step count. `PLANNER_PROMPT_v2_defensive.md` is `PLANNER_PROMPT_v2.md` plus a defensive-execution block. That block requires output validation after every per-sample step, retry-once on transient failure, per-sample isolation, a structured `results/failures.log`, a final stderr summary line, and an explicit exit-code policy.

Two pieces of content that shape the results originated in the human meta-prompt, not with the planner model, and both sit in conditions the Results treat as model-authored. The read-group string `@RG\tID:{sample}\tSM:{sample}\tLB:{sample}\tPL:ILLUMINA` appears verbatim in `PLANNER_PROMPT.md`. That prompt instructs the planner to reproduce it “EXACT[ly], character-for-character” and to warn the implementer that the separators are colons rather than equals signs. This literal string and its accompanying gotcha are therefore present in the *lean* v1 condition, not only in v2. The same meta-prompt names `lofreq call-parallel` as the required subcommand and requires `bgzip` plus `tabix` rather than `bcftools view -O z`. It also fixes `THREADS=4`, forbids duplicate marking, and requires `bcftools query` for the collapse step. The planner model contributed the ordering, the phrasing and the per-step flag selection to v1, and to v2 it additionally contributed the assembly of each step into a complete, literal invocation. The gradient is therefore, at least in part, a gradient in how much literal syntax was demanded of the planner, not purely a gradient in model-authored detail.

The single exception to model authorship is v1g, whose `lofreq` block was extracted mechanically from the Galaxy IUC tool registry (`tools/lofreq/lofreq_call.xml` at tools-iuc commit 39e7456) by `scripts/galaxy_to_snippet.py`. It is therefore a plan at v1-level density with a non-model author, and it serves as a check on the gradient rather than as an arm of a planner comparison. The first version of this manuscript called it the latter, a description withdrawn for reasons given with the result (main text, *Effects of prose and executable syntax*).

Two conditions are Track B, in which the prompt template substitutes no plan at all, so every Track-B run is a no-plan run no matter which plan file was nominally passed on the command line. Grouping runs by plan file without honouring this folds no-plan runs into the v1 and v2 columns, yielding the convincing but false conclusion that the most detailed plan performs worst. The released analysis code (`revision/scripts/gradient.py`) implements the correct mapping and asserts it. The frontier reference gradient was affected by exactly this error in the first version of this manuscript, and we recompute it here: `gradient.py --arm frontier` now generates that row, so it is derived rather than transcribed.

The executor prompt had a fixed structure, released verbatim in this Supplement: a role statement; hard constraints on the emitted script; a task specification; a dataset manifest; the tool inventory with pinned versions; the plan slot; and an output-format instruction. The hard constraints were the shebang followed by `set -euo pipefail`, all output under `results/`, idempotent re-execution, and no absolute paths outside the working directory. They also restricted the script to tools from the supplied inventory with no package managers, under a 600-s wall-clock limit at four threads. Only `v2_defensive` carries safety and validation prose; the other seven conditions do not, and that difference is a measured condition rather than an oversight.

The split also defines a data-governance boundary, and once the cost model is taken into account this boundary is the main non-cost justification for the architecture. The claim must therefore be scoped carefully: the boundary was respected by the planner role only. We prompted the planner with a workflow description, a tool inventory, generic relative paths and de-identified sample labels. No absolute filesystem path, host name, credential or read data was included in a planner prompt. The frontier *executor* arms necessarily transmitted the executor prompt. These arms are 81 API cells in the reference gradient and 234 in the Jetson error matrix (`error_matrix_anthropic.jsonl`; the first version of this manuscript wrote 117, which is the per-plan count). The executor prompt contains the dataset manifest with this study’s own subject labels (`M117-bl`, `M117-ch`, `M117C1-bl`, `M117C1-ch`, a mother–child pair), and it contains the tool inventory generated locally by `setup/verify_env.sh`. The boundary therefore holds for the deployed architecture and not for the comparator arms, which the first version of this manuscript did not disclose. The audit below shows it holds less completely for the planner role than this paragraph originally claimed.

The first version of this manuscript said the audit of what was actually sent was possible and had not been run; it has now been run, and it changes the claim. One correction first: `provider_envelope.json`, archived for each of the 81 frontier cells, is the *response* envelope — usage, cost, stop reason — and carries no request body. The earlier statement that the harness archives every request envelope is therefore wrong. The request body is instead deterministically reconstructable, because `harness/run_one.py` builds every outbound payload as `prompts/system.txt` plus one template whose only substitutions are a plan file and the stdout of `setup/verify_env.sh`. `revision/scripts/payload_audit.py` rebuilds every payload that was sent and scans it against a deny-list of local-identity tokens; Table S3 reports the result. Benchmark 2’s traffic over MCP (the Model Context Protocol, a standard interface through which a model calls external tools) is neither archived nor reconstructable and is therefore outside the audit — a gap, not a clean result.

What was actually sent to the commercial API

Table S3. Deny-list scan of the outbound payloads sent to the commercial API, by arm; cells count payloads containing at least one hit for that token class. Payloads are reconstructed rather than captured, for the reason given above.

Arm	Payloads scanned	Host name	\$HOME	Conda prefix	Absolute path	Credential	Subject label
Planner meta-prompts	3	0	0	0	0	0	3
Frontier executor gradient	81	0	81	0	81	0	81
Frontier error matrix (Jetson)	234	0	234	0	234	0	234
Benchmark 2 MCP traffic	not archived, not reconstructable	—	—	—	—	—	—

Two of the three findings were predicted and one was not. As expected, every frontier *executor* payload carries the study’s own subject labels, because the dataset manifest names them. Less expectedly, all three *planner* meta-prompts carry the same labels, because `plan/PLANNER_PROMPT.md` and its two variants state

the sample list verbatim. The labels are de-identified, so this does not contradict the paragraph above. But it does mean the planner boundary blocks paths, host metadata and read data while failing to block sample identifiers — the class a data use agreement is most likely to restrict. The only absolute path on the wire is the operator’s home directory, `/home/anton`, and it appears in every executor payload. It comes not from the manifest but from the constraint line of `prompts/system.txt`, which reads “No `/home/anton`, no `/tmp`, no absolute paths to data”. A prohibition that names the path it prohibits exports it. No host name, conda prefix or credential appeared in any payload. The harness could enforce the boundary with a pre-flight scrub of the rendered prompt against the same deny-list, refusing to send on a hit; it does not currently do so. On this evidence, the boundary held for the planner role only for paths and system metadata, not for identifiers.

Hardware sizing and measured residency

This subsection sits in Methods rather than the Introduction because it reports measurements rather than motivation. Residency figures are the `SIZE` column of `ollama ps` with the model fully resident on GPU: weights, plus the KV cache allocated for the pinned context, plus graph buffers. The figures are collected in `revision/hardware/resident_sizes.md`.

On the Jetson AGX Orin, a dense `gemma4:12b` occupying 7.6 GB generated at 14.4 tokens/s. The mixture-of-experts (MoE) `qwen3.6:35b-a3b`, occupying 23.9 GB with 3 B active parameters, generated at 30.7 tokens/s — three times the residency, 2.1 times the throughput. These are two models from two families, differing in tokenizer, layer count and architecture as well as in density. The figures come from this study’s own mixed workload rather than from a standardised throughput benchmark. The comparison supports one narrow statement: in this workload, one 3B-active MoE model outran one 12B dense model despite three times the residency. It does not support a general claim about the hardware class, and the first version of this manuscript overstated it.

We measured KV-cache growth on `qwen3.8:27b` at the three budgets of the reasoning-budget ablation, with `OLLAMA_FLASH_ATTENTION=false` and `OLLAMA_KV_CACHE_TYPE=f16`. We projected 25, 31 and 43 GB at 16k, 32k and 64k context, assuming 19 GB of weights plus a 6 GB KV cache at 16k scaling linearly. Measured residency was 25, 27 and 31 GB, so quadrupling the context added 6 GB rather than 18. Grouped-query attention, which shares key and value projections across attention heads, accounts for most of the difference. Two figures in this study are not reconcilable. Because a reader may size a purchase from them, we state the consequence first: this manuscript reports no residency figure a reader should provision against. The ablation series puts `qwen3.8:27b` at 25 GB with a 16k context, while the benchmark-2 configuration records the same model at 19.6 GB with a 65,536-token context (`revision/galaxy_demo/METHODS.md`). That is a 60% discrepancy in the direction that under-provisions, because the larger context gives the smaller number. The two were measured months apart under different service configurations, and nothing in the archived records explains the difference. Re-measuring all four models under one pinned configuration at 16k and 64k is roughly an hour of cold loads and would settle it; we did not do it in this revision. We have therefore removed the residency column from main-text Table 1 and give every residency measurement this study made in Table S4, with the discrepancy stated inline. We stand behind three statements only. A 27 B q4_K_M model occupied between 19 and 31 GB across the engine settings and contexts used here, and a 64 GB board held one at 64k context. A laboratory should measure on its own hardware rather than project from these numbers or from parameter count.

Table S4. Measured residency, and the discrepancy in it. `revision/hardware/resident_sizes.md` records every `ollama ps` measurement made in this study, with the pinned context for each; residency is weights, plus the key–value cache allocated for the pinned context, plus graph buffers. The engine was `ollama 0.32.5` with `OLLAMA_FLASH_ATTENTION=false` and `OLLAMA_KV_CACHE_TYPE=f16`, on the Jetson AGX Orin 64 GB.

Model	Quantisation	Pinned context	Resident	Source
<code>qwen3.6:27b</code>	<code>q4_K_M</code>	16,384	~17 GB	this study’s hardware notes
<code>qwen3.6:35b-a3b</code>	<code>q4_K_M</code>	16,384	23.9 GB	<code>revision/galaxy_demo/METHODS.md</code>
<code>qwen3.6:35b-a3b</code>	<code>q4_K_M</code>	65,536	31 GB	<code>ollama ps</code>
<code>qwen3.8:27b</code>	<code>q4_K_M</code>	16,384	25 GB	budget-ablation series

Model	Quantisation	Pinned context	Resident	Source
qwen3.8:27b	q4_K_M	32,768	27 GB	budget-ablation series
qwen3.8:27b	q4_K_M	65,536	31 GB	budget-ablation series
qwen3.8:27b	q4_K_M	65,536	19.6 GB	revision/galaxy_demo/METHODS.md — does not reconcile with the row above
gemma4:12b	q4_K_M	16,384	7.6 GB	this study's hardware notes

The two `qwen3.8:27b` rows at a 65,536-token context differ by 11 GB; the measurements were made months apart under different service configurations, and nothing in the archived records explains the difference. No residency figure in this paper should be used to size a purchase. Re-measuring the four models under one pinned configuration at 16k and 64k is roughly an hour of cold loads, and it was not done. Ten of the twelve benchmark-1 models were never measured at all, and no figure is estimated for them. The first version of this manuscript said eight, which is wrong, and the source file has been corrected to match.

Inference settings, sampling parameters, and the token budget

All local generations were served by ollama 0.32.5 at `temperature = 0.2` with `num_ctx = num_predict = 16384`, and for local models an integer seed was passed to the sampler. At fixed decoding parameters the seed therefore selects a trajectory through the distribution those parameters define. For the Anthropic API runs the harness constructs no seed argument at all, so the integer recorded in those rows is a bare run identifier and is reported as such. Neither case gives bitwise determinism, because batched GPU inference does not guarantee identical results even at a fixed seed, and we have removed all claims of reproducible sampling.

A confound found during revision is disclosed here in full. The harness overrode only `temperature`, leaving `top_k` and `top_p` at each model's own defaults, and those defaults are not uniform across families. `top_k` is 20 for the Qwen 3.5 and 3.6 models, 64 for Gemma 4, and unset elsewhere, while `top_p` is 0.95 for those Qwen models, Gemma 4 and GLM, 1 for Nemotron, and unset elsewhere. The family label is not a safe shorthand: `qwen3.8:27b`, the benchmark-2 dense arm, declares neither, so it ran at the engine defaults, while its predecessor `qwen3.6:27b` ran at `top_k 20 / top_p 0.95`. We therefore compared models under different sampling configurations; per-model values are in Supplementary Table S8. This is a confound and not merely a disclosure. Every cross-model comparison in this paper — the gradient, the repeated-sampling table, the abstract's contrast between arms — compares models decoded from different distributions. The model that fails at the headline condition (`nemotron-3-nano:4b`, 1/10) and the models that pass are not decoded alike. Repair does not require re-running the whole matrix. Re-running the headline cell (12 models $\times$ 10 seeds) and the discriminating cell (12 models $\times$ 3 seeds) with `top_p` and `top_k` pinned is 156 cells, roughly 4 GPU-hours against the 75.27 h already spent. It would put the pinned numbers in the headline, with the current run as a sensitivity check; that was not done in this revision. Until it is, the ordering of models under a pinned decoder is unknown, and any cross-model ordering claimed here must be read as provisional. The gradient itself is a within-model comparison across plan conditions and is not affected, because each model's decoder is constant across its own columns.

The 16,384-token budget was never justified in the original code or manuscript. It is uniform across all local models, which makes the comparison internally fair, and it is generous for the target artefact, a bash script of roughly 100 lines. It is nonetheless between one eighth and one sixty-fourth of the native context these models declare, which ranges from 131,072 tokens for `gpt-oss:20b` and both Granite models to 1,048,576 for `nemotron-3-nano:30b-a3b` (Supplementary Table S8). The first version of this manuscript stated a single 262,144-token native context, which is correct only for the Qwen, Gemma and laguna subset. The ~2.4k-token prompt is charged against the budget, leaving about 14k tokens of usable generation. Because the budget decided at least one result, the ablation reported in the Results was run as a sensitivity check.

A fixed budget can convert a verbose reasoner into an apparent failure. The harness therefore records a generation-outcome class for every cell alongside its score: `ok`, `truncated_in_thinking`, `truncated`,

`empty_content`, `provider_error`, or `refusal`. Reasoning traces are persisted separately, because ollama ≥ 0.32 returns them in a distinct field. A scorer reading only message content records a model that spent its budget reasoning as a total failure. Reasoning-enabled generations were additionally bounded by a 2,700-s wall-clock timeout, recorded as `provider_error` when it fired.

Execution environment and safety guardrails

In benchmark 1 we executed generated scripts on the same machine that hosted the model. Each script ran in a per-cell working directory, under a conda environment with a pinned tool inventory, and was killed at 600 s of wall clock. We did not review generated code before execution, and there was no container, virtual machine, seccomp profile, user namespace, cgroup quota, or network-egress restriction; scripts ran as an unprivileged user. No privilege-escalation attempt was observed. But we performed no audit of available escalation paths on the JetPack image — sudoers entries, docker group membership, writable setuid binaries on PATH. The first version’s claim of “no route to elevation” is therefore withdrawn.

Enforced and requested constraints must be distinguished. The 600-s kill, the PATH-restricted tool inventory and the working directory are enforced by the harness. In contrast, “No absolute paths outside the working directory”, “no package managers”, and “no `curl` or `wget`” are prompt instructions a model is free to violate. The first version of this manuscript said only that no destructive or escape behaviour was observed. We now measure the violation rate over every archived script and report it below (*What the emitted scripts actually did*); that measurement is the empirical content the earlier claim was standing in for.

Benchmark 2 supplies the contrasting production case: execution took place on usegalaxy.org, where jobs run in per-tool containers under the server’s own scheduler and quotas. The isolation claim in the first version of this manuscript — “the blast radius of any error is one history” — was wrong. The correction matters because this is the manuscript’s answer to what a production sandbox looks like. The executor held an ordinary Galaxy API key, which is account-scoped, not history-scoped. A holder of such a key can list, modify, purge and share every history, dataset and workflow belonging to that account, and can also create new histories and consume the account’s quota. The blast radius was therefore the account, not the history. What narrowed it in practice was the MCP server, which exposed a bounded set of verbs — import a workflow by TRS identifier (the registry address through which Galaxy imports a published workflow), build a collection, invoke, poll, read a dataset — rather than the full set of Galaxy API operations. A real deployment should narrow the blast radius deliberately rather than incidentally: a per-run service account, a dedicated history with a scoped key, and an MCP server exposing only the invoke/poll/read verbs.

Sandboxing is a prerequisite for production use of this pattern. That means a container or VM boundary, an explicit filesystem scope, no network egress by default, no `sudo`, and cgroup resource limits, plus either human review or a policy layer between generated commands and the shell. The local arm of this study does not meet that bar and is not a deployment template.

A bounded tool set must bound inputs as well as verbs, and this study exercised the input channel without controlling it. Attacker- or accident-controlled text reached the executor through at least five channels: dataset and history names on a shared multi-user public server; job stderr; tool output read back through MCP; the descriptions of the MCP tools themselves; and, in benchmark 1, 200 lines of adversarially shaped stderr injected deliberately through a PATH shim. The one fabrication incident documented in this study is an instance of the same class from an unexpected direction: the model harvested a plausible tool name out of the harness’s own injected system context and used it in a report. None of these channels was sanitised, filtered, or marked as untrusted, and no indirect-prompt-injection test was run. A deployment needs to treat tool output as data rather than instruction, and it must not let observed text authorise a state-changing call without read-back verification. That is one of the three recommendations this paper derives from its own failure modes.

Scoring, and how it departs from published practice

Table S5. Scoring choices in benchmark 1, the closest published convention for each, and the departure; this table replaces the promise of an enumeration that the first version of this manuscript made and did not keep.

Scoring choice here	Closest published convention	Departure, and why
M1/M2/M3 ladder (executed / schema-valid / matched, matching the released scorer's field names)	BixBench final-answer scoring; Biomni task rubrics	The ladder separates plumbing failure from wrong answer, which a single pass/fail cannot. Departure: no partial credit for reasoning quality, only for output.
Per-run pass as the unit: tolerant Jaccard averaged over four samples, scored 0 on non-zero exit status	GA4GH germline benchmarking, which scores per variant across a stratified truth set (Krusche et al. 2019)	The unit here is the run, not the variant, because the object under test is whether a pipeline was constructed, not how well a caller performs.
No per-variant tolerance beyond ± 0.02 on allele frequency, and no confident-region stratification	hap.py with GA4GH stratifications	The caller is deterministic and the reference is 16,569 bp: a correctly wired pipeline reproduces the answer key exactly, so stratification would have nothing to separate. 0 of 324 runs produced any allele frequency outside the window.
Truth set is the canonical output of the same caller, not an orthogonal call set	Genome in a Bottle-style external truth (Krusche et al. 2019)	No held-out truth set exists for these downsampled samples. The consequence is that precision is 1.000 by construction and the metric measures orchestration, not variant-calling accuracy.
Success proportions carry Clopper–Pearson exact intervals; nothing is pooled across models	Agent benchmarks generally report a single accuracy figure	Between-model dispersion here is gross (nine models at 10/10, one at 1/10), so a pooled proportion describes no quantity of interest.

Error injection

We injected tool failures with PATH shims. Before each error-matrix cell the harness wrote short wrappers for **bwa** and **lofreq** into a per-run directory and prepended that directory to PATH after conda activation, so the shell resolved the wrapper first. Each shim read `EVAL_INJECT_PATTERN` and `EVAL_INJECT_TARGET`, passed untargeted invocations straight through to the real tool, and applied the pattern only to the targeted one. The generated script observed nothing but ordinary exit codes, stderr and output files, and we assigned three handling metrics from those observations. The first is a handle category with four values: **crash**, an error-propagation value, **partial**, and **recover**. The second is a binary recover flag, scored against the best outcome each pattern permits, and the third is a binary diagnose flag, recording whether the script announced the failure at all.

Two of the seven patterns do not inject an error at all; Table S6 recategorises the set on that axis.

Table S6. Injected patterns, by target and by whether the tool actually fails. We wrote shims for two targets, **bwa** and **lofreq**, so five true-error patterns $\times$ two targets would be ten combinations. Two are absent by design and not by post-hoc exclusion: **silent_truncation** and **wrong_format_output** both act on the VCF, and **bwa** emits no VCF, so neither pattern is expressible against it. That is why the true-error denominator is 8 and not 10, a point the first version of this manuscript did not state.

Pattern	Targets run	Shim behaviour	Class
flake_first_call	bwa , lofreq	Exits 1 once, then passes through	true error
one_sample_fails	bwa , lofreq	Exits 1 for one named sample only	true error
missing_lib_error	bwa , lofreq	Exits 127 without running the tool	true error
silent_truncation	lofreq only (no VCF from bwa)	Runs the tool, then truncates the VCF to zero bytes	true error
wrong_format_output	lofreq only (no VCF from bwa)	Runs the tool, then strips all variant lines	true error
slow_tool	bwa , lofreq	Sleeps 30 s, then succeeds	latency tolerance, not an error

Pattern	Targets run	Shim behaviour	Class
<code>stderr_warning_storm</code>	<code>bwa, lofreq</code>	Emits 200 warnings, then succeeds	log-noise tolerance, not an error

Benchmark 2: protocol and enabling conditions

The second benchmark re-quantified *Candidozyma auris* RNA-seq from Santana et al. (2023), BioProject PRJNA904261, six paired-end runs SRR22376027–SRR22376032, against a published benchmark with known expected values (~5,594 genes, ~92% assigned). Benchmark 2 varies four things at once relative to benchmark 1: a second workflow class, a second infrastructure — a shared multi-user production server rather than a private board — and a 30-step workflow rather than a 7-step script. The fourth difference is the existence of an external cost comparator (Nekrutenko 2026). It is a case study, with one invocation per arm, two executors, no replicates, no frontier control (no API-hosted model run alongside for comparison), and no interval on anything.

The executor-selection rule is invoked three times in this manuscript and was never given, so we state it in full here.

Take every local model that scored 3/3 at plan v2 Track A with reasoning off in the benchmark-1 gradient. From that set, take the largest dense model and the largest mixture-of-experts model by total parameter count. The two benchmark-2 arms then differ in architecture and not only in size.

Applied to the twelve-model set frozen before benchmark 2, the rule returns `qwen3.6:27b` (dense) and `qwen3.6:35b-a3b` (MoE). The dense arm actually run, however, was `qwen3.8:27b`, substituted for its predecessor at matched parameter count and quantisation. That model was released on 14 August 2026, after the rule was fixed and after benchmark 1 was complete. The substitution is a deviation from the frozen rule, so the dense arm is exploratory rather than confirmatory. The first version of this manuscript described four elements as frozen, and three survive as confirmatory: the selection rule itself, the reasoning-off setting, and the hardware. The model selection does not survive, and plan sufficiency was never confirmatory on either benchmark. The Galaxy plan was corrected repeatedly while benchmark 2 ran — eleven times by the author’s contemporaneous record in `revision/galaxy_demo/FACTS.md`, a count the released artifacts cannot confirm.

Both executors ran on one Jetson AGX Orin 64 GB with reasoning suppressed in both. The dense arm was `qwen3.8:27b` (19.6 GB resident) and the MoE arm was `qwen3.6:35b-a3b` (23.9 GB resident, ~3 B active). We ran three arms in total: a human validation arm, in which a person drove the same workflow manually, plus one arm per executor. All three history and invocation identifiers are listed in the Supplement.

The harness was `galaxyproject/loom`, with Galaxy reached over MCP and `notebook.md` serving as durable inter-session state. The plan was a router: a 50-line index plus seven step files, so the executor loads only the step it is on. The loop had seven stages. It imports the IWC `rnaseq-pe` v1.5 workflow (`fastp` → `STAR` → `featureCounts`) by its TRS identifier, then locates and verifies inputs, builds or verifies the paired collection, and resolves the annotation. It then constructs and submits a 30-step invocation, polls invocation and job state, and reports. Unlike benchmark 1, every turn produced an observation the model read before deciding what to do next.

We report several enabling conditions because they are part of the result. Context was pinned to 65,536 tokens via a Modelfile (ollama’s per-model configuration file), because at the model default ollama reserves the full native context and allocates 53 of 61 GB. We routed requests through a LiteLLM proxy using the `ollama_chat/` provider for one reason: ollama’s OpenAI-compatible `/v1` shim silently drops the `think: false` flag, while the native `/api/chat` endpoint honours it. A fresh-session flag and a hard `TERM` timeout were needed because the Galaxy poller keeps headless runs alive.

No number reported for this benchmark comes from an agent’s own report: every state, parameter and count was read back from the Galaxy API or from dataset contents. The reason is that one executor produced a fully formed failure report for an event that never occurred, and the only thing distinguishing it from a true report was the API. That read-back was performed live, and we transcribed its outputs into

revision/galaxy_demo/FACTS.md and featurecounts_summary.csv; the raw jobs-summary JSON was not archived. Job-level outcome counts appeared in the first version of this manuscript and are removed here, because no released file supports them.

Scoring had four components. Components (a) and (b) are invocation correctness, checked field by field against the validated invocation, and biological outcome: gene count and per-sample assigned fragments against the published benchmark, read from dataset contents. Component (c) is end-to-end reporting — whether the executor itself delivered the required per-sample table with a dataset identifier beside every number. Component (d) is a defect ledger attributing every human intervention to the planner side or the executor side.

One methodological change belongs in Methods because it changed the design. The original step-7 verification file stated the expected values, and that lets an executor that never opens a dataset produce a flawless verification by reciting the task. We rewrote the file to state no expected values, to require a dataset identifier beside every reported number, and to sanction **UNAVAILABLE** as an answer. The general point: any agent evaluation that places the expected answer in the task description cannot distinguish execution from recitation.

The fabrication-rate study

The fabrication incident was a single observation, so we ran a controlled follow-up to bound the rate. The design was 3 conditions $\times$ 2 models $\times$ 5 seeds, with one fresh workspace per cell, a fixed prompt, and known ground truth; the design is therefore 30 launches. The task was to fetch six featureCounts summary datasets from a real Galaxy history and report each **Assigned** value with the dataset identifier it was read from. Twenty-four of the 30 launches produced a scorable answer file, 13 for the dense model and 11 for the MoE model. That gives 78 and 66 sample-level judgements respectively, 144 in total. Six of the 30 launches, 20%, produced no answer to score. The first version of this manuscript said the count of excluded launches could not be quantified; it is recoverable from the design statement and is given here. The per-model and per-condition split of those six is not recoverable, because the released records do not identify which launches they were.

Each condition varied one thing. The blind condition stated nothing about expected values, and the leaky condition stated the expected range, exactly as the original step-7 file did. The blocked condition replaced two of the six dataset identifiers with same-shaped identifiers that do not exist (verified to return HTTP 400). Those two values cannot be obtained, so **UNAVAILABLE** is the only correct answer for them. We scored answers into four categories chosen to separate silence from a claim: **CORRECT** is the true value, and **WRONG** is any other number — the fabrication category. **ABSTAINED** is an explicit **UNAVAILABLE**, and **ABSENT** is no statement at all. In the blocked condition an abstention on an unreadable identifier is scored correct, and a number reported there is scored as a fabrication.

Three limitations govern how the rate may be read. First, the six judgements within a session are not independent: a session that opens the right datasets gets all six right, and one that does not gets none right. The session, not the judgement, is therefore the unit of analysis, and we report both denominators. Second, launches producing no answer file are not scored at all. The observed mode was a session that made zero tool calls, ran about a minute, emitted a single malformed pseudo-citation and stopped. The rate therefore measures fabrication conditional on an answer having been produced, at a 20% no-answer rate, and the excluded class is structurally the one the original incident belongs to. Third, the incident arose in a resumed session carrying roughly 27,000 tokens of prior work with no reachable tool path, while every cell here is a single-shot task with a working path. The study therefore does not reproduce the regime in which fabrication occurred.

Platforms, engines, and reproducibility metadata

The study ran on one platform (Table S7); runs from four earlier platforms exist in the repository but are out of this paper’s scope. A general engine fact is still worth recording for anyone re-running the evaluation elsewhere: identical weights do not give identical outputs across machines. Part of the reason is that the

distributed builds are not the same build of the same weights (GGUF q4_K_M under ollama on CUDA devices; MLX 4-bit on Apple silicon), with different kernels and batching.

Engine configuration on the Jetson affects the results directly. We disabled flash attention, and ollama itself warns that it cannot verify its compiled CUDA architectures for sm_87. With flash attention enabled we recorded 22 illegal-memory-access faults in six hours across 5 of 12 models; after disabling it we recorded zero faults in roughly 41 h. The key-value cache was set to f16 rather than q8_0, and one model was loaded at a time.

Supplementary Table S8 is generated by script and records, per model, the exact tag, blob digest, architecture, parameter count, quantisation format, context length, declared capabilities and default decoding parameters. It also records the engine version (ollama 0.32.5), the platform (JetPack 5.1.2, CUDA 11.4, compute capability 8.7) and the pinned analysis tools (bwa 0.7.18, samtools 1.21, bcftools 1.21, htslib/tabix 1.21, lofreq 2.1.5). A data hazard affecting an out-of-scope platform’s archived logs is recorded once, at *Reproducibility hazard* below, for anyone reusing the repository.

Per-cell provenance is partial, and this is a reproducibility limitation of the released artifacts rather than of the study. Each cell’s `meta.json` records the plan file path and plan name it was rendered from, but not a git revision for that file. The released data therefore supports attributing a generated command to a plan *file* and not to a plan *revision*. Plans are versioned in git, and the defect ledger attributes corrections to specific revisions, so the link is recoverable by hand; it is not machine-checkable from the per-cell records. Adding a plan blob SHA to `meta.json` would close this gap, and that was not done.

Table S7. The platform, with the inference engine and quantisation used. Earlier drafts reported four further machines (an RTX 5080 desktop, two Apple laptops and a 2× RTX A5000 workstation); their runs are retained in the repository but are out of this paper’s scope, because only the Jetson dataset is complete across all conditions and passes.

Platform	Maker	Year	Memory	OS	GPU	Engine / quantisation
Jetson AGX Orin Developer Kit	NVIDIA	2022	64 GB LPDDR5 unified	Ubuntu 20.04.6, JetPack 5.1.2 (L4T 35.4.1)	Integrated Ampere, 2,048 CUDA cores	ollama 0.32.5, GGUF q4_K_M

Table S8. Reproducibility metadata.

Collected by `revision/scripts/collect_repro_metadata.py` into `revision/repro_metadata.json` and rendered with its `--markdown` flag, so the table is regenerated rather than transcribed. Digests are the ollama manifest blob digests as resolved on the Jetson at collection time.

This table contains all fourteen local models the study uses. The first version of it contained twelve. It omitted `gemma4:12b`, used for a throughput measurement only, and it omitted `qwen3.8:27b` — the benchmark-2 dense arm, the out-of-sample row in Figure 2 and the subject of the entire reasoning-budget ablation. The second omission mattered. This manuscript’s own disclosure is that `top_k` and `top_p` were left at publisher defaults, which differ across families, so omitting that row left the sampling configuration of the case-study executor unknown to a reader. We re-ran the collection script against both models, which were still resident on the board. The two rows are now present, and the last twelve rows reproduce the earlier collection digest for digest. Two corrections came out of the re-run. First, `qwen3.8:27b-q4_K_M` declares no default `top_k` or `top_p` at all, so the case-study executor was decoded at the engine defaults rather than at a publisher setting. That is the one configuration this manuscript could not previously state. Second, the collector had a parsing fault that this model exposed: a multimodal tag carries a `Projector` block whose own `architecture` and `parameters` keys were being filed as the model’s, so `qwen3.8:27b` first collected as `clip` at 460.73 M. The fault is fixed in `collect_repro_metadata.py`, and it changed none of the twelve original rows. We record it here because a script-generated table is only as trustworthy as its parser. The model’s measured residency (19.6 GB at a 65,536-token context) is in `revision/galaxy_demo/METHODS.md` and Table S4, and it does not reconcile with the ablation series.

Model tag	Digest	Architecture	Parameters	Quantisation	Context length	Capabilities	Publisher default decoding parameters
laguna-xs-2.1:0478b41e57f4	0478b41e57f4	laguna	33.4B	Q4_K_M	262144	completion, tools, thinking	none declared
qwen3.6:35b-a307d45712591f	307d45712591f	qwen35moe	36.0B	Q4_K_M	262144	completion, vision, tools, thinking	temp 1; top_k 20; top_p 0.95; min_p 0; repeat 1; presence 1.5
gemma4:26b-a4b5571076f3070	4b5571076f3070	gemma4	25.8B	Q4_K_M	262144	completion, vision, tools, thinking	temp 1; top_k 64; top_p 0.95
nemotron-3-nanb720b1a35407_KnM	720b1a35407_KnM	nemotron_h_m3d.6B	34.6B	Q4_K_M	1048576	completion, tools, thinking	temp 1; top_p 1
gpt-oss:20b	17052f91a42e	gptoss	20.9B	MXFP4	131072	completion, tools, thinking	temp 1
glm-4.7-flash:4475827791a2	4475827791a2	glm4moelite	29.9B	Q4_K_M	202752	completion, tools, thinking	temp 1; top_p 0.95; min_p 0.01; repeat 1
qwen3.6:27b-q4a50mda8ed977	4a50mda8ed977	qwen35	27.8B	Q4_K_M	262144	completion, vision, tools, thinking	temp 1; top_k 20; top_p 0.95; min_p 0; repeat 1; presence 1.5
gemma4:31b-it-0416f0629137	416f0629137	gemma4	31.3B	Q4_K_M	262144	completion, vision, tools, thinking	temp 1; top_k 64; top_p 0.95
granite4.1:30b363e5d78a021	63e5d78a021	granite	28.9B	Q4_K_M	131072	completion, tools	none declared
granite4.1:8b-444xf1c4b2fe	444xf1c4b2fe	granite	8.8B	Q4_K_M	131072	completion, tools	none declared
qwen3.5:4b-q4_2a054d98e6fb	2a054d98e6fb	qwen35	4.7B	Q4_K_M	262144	completion, vision, tools, thinking	temp 1; top_k 20; top_p 0.95; presence 1.5
nemotron-3-nan6c467f05439	6c467f05439	nemotron_h	4.0B	Q4_K_M	262144	completion, tools, thinking	temp 1; top_p 1
qwen3.8:27b-q475322a6ba435	475322a6ba435	qwen35	26.9B	Q4_K_M	262144	completion, vision, tools, thinking	none declared
gemma4:12b	4eb23ef187e2	gemma4	11.9B	Q4_K_M	262144	completion, vision, audio, tools, thinking	temp 1; top_k 64; top_p 0.95

Two facts in this table matter for interpretation. First, quantisation is not uniform: `gpt-oss:20b` is MXFP4 and every other local model is Q4_K_M, which is why main-text Table 1’s caption states the exception rather than claiming uniformity. Second, the publisher defaults for `top_k` and `top_p` differ across families: `top_k` is 20 for the Qwen 3.5 and 3.6 models, 64 for the Gemma 4 family, and unset elsewhere. The value of `top_p` is 0.95 for those Qwen models, Gemma 4 and GLM, 1 for Nemotron, and unset elsewhere. `qwen3.8:27b` declares neither, so the family name does not predict the setting. The harness overrides only temperature, so models were compared under non-identical sampling configurations. We disclose this rather than correct it, because correcting it costs a full re-run of the matrix. Declared context lengths range from 131,072 to 1,048,576 tokens. That range is the basis for the statement in Methods that the 16,384-token budget is between one eighth and one sixty-fourth of native context.

The harness overrode `temperature` to 0.2 and set `num_ctx = num_predict = 16,384` for the plan-gradient matrix, leaving `top_k` and `top_p` untouched. For benchmark 2 the context was pinned to 65,536 through a Modelfile, because at the model default ollama reserves 53 of the board’s 61 GB.

The engine was ollama 0.32.5 on JetPack L4T R35 revision 4.1 (JetPack 5.1.2, CUDA 11.4), running on an NVIDIA Jetson AGX Orin 64 GB at compute capability 8.7. The service environment sets `OLLAMA_FLASH_ATTENTION=false` (the flash-attention kernels are miscompiled for sm_87 on this build and produced 22 illegal-memory-access faults in six hours across 5 of 12 models; none in the ~41 h after disabling). It also sets `OLLAMA_KV_CACHE_TYPE=f16` (q8_0 is lossy and alters output), `OLLAMA_MAX_LOADED_MODELS=1`, and `OLLAMA_NUM_PARALLEL=1`.

The pinned analysis tools (conda environment `bench`, linux-aarch64) are asserted before every run by `setup/verify_env.sh`, whose stdout is the tool inventory injected verbatim into the prompt. The inventory is bwa 0.7.18, samtools 1.21, bcftools 1.21, tabix/htslib 1.21, lofreq 2.1.5, SnpSift 5.2, snpEff 5.2, fastqc 0.12.1, seqkit 2.8, snakemake 8.20, shellcheck 0.10, java 21.

Reproducibility hazard (repository note; out-of-scope platform). One archived error-matrix log, `error_matrix_ollama_m4.jsonl`, has a reduced schema in which the `wall_s` field means something else and reads about 2 s; the `_r2` variant of the file carries the correct timings. No result in this paper uses either file — the note exists so that repository users do not regenerate nonsense from the obvious filename.

Cost model

Two of the model’s eight inputs are measured: a frontier-API cost of \$0.0662 per run, the mean over 81 API cells, and a Jetson time per run of 86.7 s. The time figure was recorded for the block at run time (7.8 h over 324 cells, including model loading, execution and scoring) but was not archived, so it is a stated input rather than a verifiable measurement. The archived log supports 80.3 s per run of generation alone (26,010 s over 324 cells). Substituting 80.3 s changes the electricity term by 8% and no conclusion.

The first version of this manuscript reported break-even against a three-year cost of ownership, a comparison that is internally inconsistent: the horizon is three years and the break-even arrives after fifteen. Under the model’s own assumptions, that requires buying the hardware five times. The model now also reports a like-for-like annualised comparison, in which both columns are a cost per year at a stated run rate and the local column charges a hardware and setup refresh every depreciation period. The model further exposes an API-side labour term: harness setup, key and quota management, prompt maintenance. The first version charged that term entirely to the local arm and set it implicitly to zero for the API arm. Both are options on the released script (`revision/scripts/cost_model.py --api-staff-hours-year`), and the sensitivity of break-even to that term is reported with the result.

Supervision labour is not in the model. Benchmark 2 consumed supervised human time in three forms the cost model does not price. These are repeated plan correction; the manual pre-building of the input collection and the annotation; and the manual read-out of six per-sample values that neither executor reported. The wall-clock cost of the human validation arm, from first API call to counts in hand, was not recorded and cannot be recovered from the released artifacts. The one comparison that would settle the motivating question is therefore set up and not made. The cost model should be read as the zero-touch case, which benchmark 2 did not achieve.

Table S9. Cost model. Measured inputs are marked as such; the remainder are assumptions exposed as parameters in `revision/scripts/cost_model.py`.

Quantity	Value	Source
Inputs measured / assumed	2 of 8 measured	measured: API cost per run, local wall-clock time per run. Assumed: board draw under load, board draw idle, electricity tariff, hardware price, setup hours, maintenance hours per year (at a loaded \$75/h, itself an assumption)
API cost per run	\$0.0662	measured, 81 runs

Quantity	Value	Source
Local wall-clock time per run	86.7 s	recorded at run time, not archived; the archived generation-only figure is 80.3 s (verified)
Local marginal cost per run	\$0.00016	electricity only
Local fixed cost, annualised	\$1,682 / year	hardware + setup over 3 yr, plus maintenance and idle power
Annualised crossover, no API-side labour	25,475 runs / year	like-for-like; the figure to quote
Annualised crossover, 4 h/yr API-side labour	17,900 runs / year	sensitivity
Annualised crossover, 8 h/yr API-side labour	10,325 runs / year	sensitivity
Three-year cost of ownership	\$5,045	\$3,000 of it local-side labour
Runs to recover that total	76,424	15.3 years at 5,000 runs/yr; horizon-inconsistent, not a break-even
Runs to recover the 3-yr total at \$0.25 per API run	20,192	sensitivity sweep
Runs to recover the 3-yr total at \$1.00 per API run	5,046	sensitivity sweep
Runs to recover the 3-yr total at \$5.00 per API run	1,009	sensitivity sweep

Supplementary Results

Generation outcomes by pass

Table S10. Generation outcomes by pass.

Counts of cells whose generation did not complete normally, by released log file. Under the single exclusion rule (Methods, *Scoring, error injection, and interval estimates*) all of these are scored 0 and retained. The first version of this table listed six of the ten blocks, so it totalled 813 cells and 47 non-ok against Table S1’s 848 and 52. All ten blocks are now present and the totals reconcile.

Pass	Cells	ok	Non-ok	Which cells
Frontier reference gradient	81	81	0	—
Plan gradient, reasoning off	324	323	1	gpt-oss:20b v1g seed 42,
Plan gradient, reasoning on	270	252	18	truncated_in_thinking qwen3.5:4b ×13, glm-4.7-flash ×3, nemotron-3-nano:4b ×1, qwen3.6:35b-a3b ×1
Repeated sampling at v2 Track A	84	83	1	gpt-oss:20b v2 seed 45,
qwen3.8:27b, reasoning off	27	27	0	truncated_in_thinking
qwen3.8:27b, reasoning on at 16k	27	0	27	all truncated in reasoning; excluded from the eleven-model difference for that reason
Budget ablation at 32,768 tokens	3	0	3	qwen3.8:27b v1g, all
Budget ablation at 65,536 tokens	3	3	0	truncated_in_thinking

Pass	Cells	ok	Non-ok	Which cells
Reasoning-effort sweep, gpt-oss:20b	20	18	2	v1 at high effort, seeds 42 and 43, truncated_in_thinking ; the high-effort arm ran 2 of its 9 design cells and was re-run at 131k
Reasoning-effort sweep at 128k context	9	9	0	—
Total	848	796	52	reconciles with Table S1

Repeated sampling with plan v2, per model

Table S11. Repeated sampling at plan v2, Track A, reasoning off, pooled over ten seeds (42–51). Intervals are Clopper–Pearson exact 95% confidence intervals on the per-run proportion. Generation outcome is given because one non-perfect run is a plumbing failure rather than a capability failure.

Executor	Perfect runs / n	95% CI	Non-ok generations
gemma4:26b-a4b-it	10/10	[0.69, 1.00]	0
gemma4:31b-it	10/10	[0.69, 1.00]	0
glm-4.7-flash	10/10	[0.69, 1.00]	0
granite4.1:30b	10/10	[0.69, 1.00]	0
granite4.1:8b	10/10	[0.69, 1.00]	0
laguna-xs-2.1	10/10	[0.69, 1.00]	0
nemotron-3-nano:30b-a3b	10/10	[0.69, 1.00]	0
qwen3.6:27b	10/10	[0.69, 1.00]	0
qwen3.6:35b-a3b	10/10	[0.69, 1.00]	0
gpt-oss:20b	9/10	[0.55, 1.00]	1 (truncated_in_thinking , seed 45)
qwen3.5:4b	7/10	[0.35, 0.93]	0
nemotron-3-nano:4b	1/10	[0.00, 0.45]	0

gpt-oss:20b’s single miss is the retained generation failure described in the Methods: at seed 45 the model returned an empty script after 54,295 characters of reasoning with `done_reason=length`, despite `think: false`. Scored 0 by the retention rule, it gives 9/10; on completed generations gpt-oss:20b is 9/9 ([0.66, 1.00]). The corresponding failure in the gradient pass is at v1g seed 42.

Conventional variant-calling metrics by plan condition

Table S12. Conventional variant-calling metrics by plan condition.

Computed over all 324 reasoning-off runs by `revision/scripts/variant_metrics.py` into `revision/variant_metrics_thin`. Precision is 1.000 in every condition by construction, because the truth set is the canonical output of the same deterministic caller. Recall does not equal the run-level success rate, and the statement that it does has been withdrawn from this manuscript. The two are compared column by column below and differ in every condition except v2. The gap is real biology that the run-level metric discards through its exit-status rule. It comes from 26 runs that produced 2 of 9 truth variants and 8 runs that produced all four VCFs and then exited non-zero (main text, Results). This gap is the most informative thing in this table; the first version reported the table for completeness only. No figure here should be quoted without the conditions it pools over, which is why the pooled row is given last rather than first.

Condition	Runs	TP	FP	FN	Precision	Recall	Run-level success rate	Gap
Track B (no plan)	108	84	0	888	1.000	0.086	0.056 (6/108)	+0.030
v0.5	36	15	0	309	1.000	0.046	0.028 (1/36)	+0.018
v1	36	99	0	225	1.000	0.306	0.250 (9/36)	+0.056

Condition	Runs	TP	FP	FN	Precision	Recall	Run-level success rate	Gap
v1g	36	94	0	230	1.000	0.290	0.250 (9/36)	+0.040
v1.25	36	157	0	167	1.000	0.485	0.417 (15/36)	+0.068
v1.5	36	287	0	37	1.000	0.886	0.833 (30/36)	+0.053
v2	36	306	0	18	1.000	0.944	0.944 (34/36)	0.000
Pooled over all conditions	324	1,042	0	1,874	1.000	0.357	0.321 (104/324)	+0.036

Rows in `variant_metrics_thinkoff.json` carry the model and plan column but no per-run identifier, so one-row-per-run provenance cannot be audited from the file alone; we re-derived the totals above from the run logs. The pooled F1, 0.527 in the first version of this manuscript, is deleted: it averages over conditions that share no denominator of interest and re-expresses a binary.

The detection accounting closes exactly: truth is 9 variants per run, and $324 \times 9 = 2,916 = \text{TP} + \text{FN}$. The *run-level* accounting does not close, and the first version’s claim that it closed exactly concealed the discrepancy: 110 runs recovered all nine truth variants, while 104 scored a perfect run. The 6 missing runs wrote every VCF and then exited non-zero, so the exit-status rule set their score to 0 (*Scoring*, above). Separately, 26 runs recovered exactly 2 of 9 variants: 15 at Track B, 3 at v0.5, 2 at v1g, 2 at v1.25 and 4 at v1.5. Allele-frequency error on true positives is 0.0000, and 0 of 324 runs produced any allele frequency outside the ± 0.02 window.

What the emitted scripts actually did against the requested constraints

The prompt asks for constraints the harness does not enforce, and the rate at which they were violated is a grep over released artifacts rather than an impression (`revision/scripts/script_audit.py`). The corpus is every `run.sh` archived under `revision/runs/` — 1,039 scripts. The original 766 cover all archived cells of the frontier reference gradient, both plan-gradient arms, the repeated-sampling pass and both `qwen3.8:27b` passes, less the 47 cells whose generation produced no script. To them the audit adds the 273 scripts of the seed extension. The budget-ablation and reasoning-effort blocks are archived elsewhere and are outside it (Table S2). The audit is lexical over script text: it reads what the script says, not what it would do, and it therefore cannot see a command assembled at run time. That is why we report a count of dynamic dispatch beside the others rather than leave it out.

The exact counts follow. None of the 1,039 scripts invokes `curl`, `wget`, `pip`, `conda`, `mamba`, `apt`, `yum` or `npm` (0 of 1,039), and none invokes `sudo`, `su` or sets a `setuid` bit (0 of 1,039). Writes to a path under `/tmp/`, which is outside the per-cell working directory, occur in 19 of 1,039 scripts (1.8%). A scan targeted at redirection targets and output-flag arguments finds 0 of 1,039 directing output to an absolute path outside `/dev/` and `/tmp/`; this is the second requested constraint the audit can count exactly, and it was not violated. A recursive `rm` appears in 14 of 1,039 scripts, and 4 of 1,039 contain a recursive or forced `rm` whose target is neither under `results/` nor a script-created temporary directory — all four are quoted below. The auditor’s own `outside-results/` counter reports 0 here because its target test treats `/tmp/tmp*` as a script-made temporary path, so the adjudicated count supersedes it. Dynamic dispatch — `eval`, `bash -c`, `sh -c`, `python -c`, or a substitution feeding an executed command — appears in 3 of 1,039. One builds `bwa mem` and `bcftools query` invocations as strings and dispatches them with `eval`, a second does the same for `bwa mem` and `lofreq`, and the third wraps a sample loop in `parallel ... bash -c`. We read all three, and their constructed commands stay within the pinned inventory, so every other count here is a count over scripts that say what they do. The only binaries invoked that are not in the pinned inventory are the case-variant misspellings `snpSift` and `snpstift` (the inventory provides `Snpsift`), in 10 script lines; they fail rather than execute.

Two counters in the released auditor are noisy, and the first version of this manuscript reported neither, on the tool’s own grounds — a filter chosen after seeing the number. We report both here with their adjudication. `abs_path` fires on 240 of 1,039 scripts, 592 occurrences. We hand-classified the 479 occurrences in the original corpus: 418 are the tail of a `${VAR}/...` expansion resolving inside the working directory; 50 under `/tmp/`; 3 `/dev/stdin`; and 8 awk regular-expression fragments or `results=$(pwd)/results`. We reviewed the 113 occurrences added by the seed extension the same way: shebang lines, `/dev/` targets, two `/tmp/` paths, three awk fragments and one `read` of a mistyped absolute reference path. None in either set is a write to an absolute path outside the sandbox, which is what the targeted scan above reports as 0 of

1,039. `write_outside_results` fires on 766 of 1,039, dominated by redirections into shell variables that hold `results/` paths; the targeted scan supersedes it, and it is not quotable as a violation rate.

The first version’s sentence “No destructive or escape behaviour was observed” is replaced by an adjudicated count. Three scripts issue a delete whose target is outside `results/` and outside a script-created temporary directory:

```
find "$TMP" -mindepth 1 -maxdepth 1 -exec rm -rf {} +           # $TMP is script-created
rm -f /tmp/*.txt /tmp/temp_*.vcf.gz 2>/dev/null || true       # unbounded wildcard in a shared directory
rm -f /tmp/${sample}.unsorted.bam
rm -rf /tmp/tmp* 2>/dev/null || true                          # unbounded recursive wildcard, shared d
```

The first is bounded by construction, and the third names a single file. The second and fourth are genuinely out-of-sandbox actions: each is an unbounded wildcard delete against `/tmp/`, a shared multi-user directory, and the fourth is also recursive. On a machine with other users’ scratch files, either would delete them. The second came from a no-plan run; the fourth came from a plan-conditioned run (`qwen3.5:4b`, `v1g`, seed 47, in the seed extension). The word “recursive” in the first version’s phrasing rescued its sentence on a technicality no reader would apply.

The `/tmp/` rate is informative about plans rather than about safety: the writes are more frequent in the no-plan condition, 11 of 303 no-plan scripts (3.6%) against 8 of 736 scripts written with a plan (1.1%). A model given no plan invents its own scratch-space convention, and that convention leaves the sandbox; a plan that names `results/` as the output root lowers the rate. It does not eliminate the class: one of the two out-of-sandbox deletes came from a plan-conditioned run.

The reasoning arms in full

A plan and a chain of thought both supply intermediate steps, so it is reasonable to expect one to substitute for the other. We therefore ran both arms of the plan gradient — the same models and plans with reasoning disabled and then enabled. The disabled arm comprises 324 cells (1 non-ok generation, 0.3%) and the enabled arm 270 cells (18, 6.7%), with zero CUDA faults across 37 h (Figure 4). We restrict the comparison to the 10 models runnable in both arms: both Granite models are excluded because Ollama rejects the `think` parameter outright (HTTP 400).

Reasoning helps where the plan is thin and costs accuracy where the plan is already sufficient (Figure 5). Under the manuscript’s stated convention — truncated generations are scored 0 and retained — the per-plan differences (enabled minus disabled) over the ten both-arm models follow. Track B is +0.02; `v0.5` is +0.07; `v1` is +0.17; `v1g` is +0.03; `v1.25` is +0.23; `v1.5` is +0.07; and `v2` is −0.20. The first version of this manuscript printed +0.18 at `v1` and +0.05 at `v1g`; neither reproduces under either convention. The corrected values are given here. Under the alternative convention — dropping cells with no M3 — the same vector is +0.03, +0.08, +0.23, +0.07, +0.26, +0.07, −0.20. Both vectors are emitted by `revision/scripts/gradient.py` in its per-plan difference mode (`--truncation {zero,drop}`), not computed by hand. The two conventions agree exactly at `v1.5` and `v2` and diverge everywhere else, most at `v1` (+0.17 against +0.23), then at `v1g` (+0.03 against +0.07) and `v1.25` (+0.23 against +0.26). The first version of this manuscript said they agreed except at `v1` and `v1.25`, which understates the divergence at `v1g` — the condition where the reasoning-on arm loses a cell under the drop rule. That sensitivity is itself the reason to state the convention. At `v2` Track A, 4 of 10 models regress, none improve and 6 are unchanged.

Including `qwen3.8:27b` deepens the `v2` term to −0.27, but that figure is not comparable and should not be quoted. All 27 of `qwen3.8:27b`’s reasoning-on cells at the 16k budget truncated without returning a script, so they enter the eleven-model average as zeros. The ten-model figure of −0.20 rests on cells that completed and were scored; the eleven-model figure is a budget artifact. The first version of this manuscript attached the “not a budget artifact” disclaimer to the eleven-model number, where it is false.

One caveat travels with the result. The reasoning-enabled arm carries a 6.7% rate of harness-bounded failures — generations cut off at the token budget rather than completed — against 0.3% for the disabled arm. It therefore measures model-plus-budget rather than model capability alone.

We took the model that failed hardest — `qwen3.8:27b` on plan `v1g`, Track A, reasoning enabled — through a budget curve at 3 seeds per budget. A truncation rate that high demands to know what a larger budget buys. At 16k tokens the model truncated 3/3 times with 0/3 correct at ~2,200 s per cell, and at 32k it again truncated 3/3 times with 0/3 correct at ~5,480 s. At 64k it truncated 0/3 times with 1/3 correct at 7,395–12,974 s, while the same condition with reasoning disabled at 16k is 3/3 correct in ~130 s. At 64k the model self-terminates after 35,000–55,000 tokens of reasoning, two to three times the budget the whole study uses. The 16k and 32k rows are therefore pure budget artifacts and carry no capability information. The defensible statement is narrow: enabling reasoning on this model requires a token budget two to three times larger merely to produce output. Even when that budget is supplied, accuracy falls from 3/3 to 1/3 and wall-clock cost rises 57- to 100-fold.

Two interpretations offered earlier in this work are superseded. The first was that reasoning-enabled scripts are systematically more elaborate and that the elaboration is what breaks them. In fact the median enabled/disabled script-size ratio at v2 across the other ten models is $1.00\times$, and the most elaborate script in the ablation (2,984 bytes, 8 staleness helpers) is the one that scored 1.00. The reasoning-enabled failures at v2 are ordinary bad bash: unterminated blocks, malformed pipes, a hallucinated `bwa` subcommand. The second interpretation was that the budget explanation was dead; moving from 16k to 64k moved the condition from 0/3 to 1/3, so the budget was genuinely part of the problem.

An independent probe points the same way, with one disclosure the first version of this manuscript omitted. `gpt-oss:20b` exposes a reasoning-effort control, and we ran low and medium effort at a 32,768-token budget. The high-effort arm at that budget terminated after 2 of its 9 cells, both `truncated_in_thinking` at plan v1, and was re-run at the model’s native 131,072-token context. Mean reasoning length is therefore 480, 10,053 and 139,705 characters across low, medium and high, but the span is computed across two different token budgets — exactly the confound this manuscript polices in the `qwen3.8:27b` reasoning result. The span should therefore not be quoted as a 290-fold manipulation at fixed budget. With that substitution stated, the results at the two budgets used are these: a sufficient plan (v2) is 3/3 at low, medium and high effort, and an insufficient plan (v1) is 0/3 at low, medium and high. Only the intermediate band moves, non-monotonically, with medium (2/3 at v1.25) beating both ends. Every “at every effort level” statement here rests on the 131k re-run for its high arm. The same experiment limits our own generalisation: at a generous budget `gpt-oss:20b` is 3/3 at v2 at every effort. The v2 reasoning penalty is therefore not general, and the claim narrows to `qwen3.8:27b` specifically. A roughly 12-cell re-run at generous budget would settle whether any v2 penalty survives for the others; it was not run. Finally, `think: false` is a request rather than a guarantee: most models honour it, and both Granite models reject it outright. `gpt-oss:20b` ignores it, emitting 47,606 and 60,523 characters of reasoning with the flag off.

Wall time on the Jetson

Table S13. Median wall time per generation on the sufficient plan (v2) on the Jetson AGX Orin. The row is the Jetson row of the table emitted by `revision/scripts/table6.py --markdown`; the generator also emits the four out-of-scope platforms, whose rows are omitted here. The figure is generation time — the model producing the script — and does not include script execution; the 600-s harness limit applies to execution only and does not bound generation. Both dispersion columns are emitted by the generator, which asserts $\min \leq Q1 \leq \text{median} \leq Q3 \leq \text{max}$ before printing. An earlier five-platform version of this table hand-entered its range column and got four of five rows wrong; the platforms other than the Jetson are now out of scope (Table S7), and their rows are removed rather than corrected.

Platform	n	Median generation (s)	IQR	Full range
Jetson AGX Orin	36	105	[98–107]	[97–227]

The tail matters: a generation that usually takes 105 s took 227 s once, and timing dispersion on one machine is itself a reason a per-run wall-clock budget needs headroom.

Benchmark 2: per-sample counts and the defect ledger

Table S14. Per-sample featureCounts outcome, read back from the Galaxy API and from dataset contents rather than from any executor’s report (`revision/galaxy_demo/featurecounts_summary.csv`). The dense and mixture-of-experts arms produced byte-identical values in all four featureCounts categories from twelve distinct dataset identifiers, so one set of values is given. The published benchmark of ~92% assigned corresponds to the uniquely-mapped denominator.

Run	Assigned fragments	% of uniquely mapped	% of all fragments	Multi-mapped (%)
SRR22376027	25,270,576	93.3	84.0	10.0
SRR22376028	18,286,752	93.4	83.8	10.2
SRR22376029	15,721,261	93.2	76.6	17.9
SRR22376030	21,231,776	93.1	77.4	16.9
SRR22376031	20,130,414	93.6	85.2	9.0
SRR22376032	22,162,227	93.5	83.1	11.1

Repeated human correction of the plan, and not any change of model, is what produced a clean run. The count and its attribution come from `revision/galaxy_demo/FACTS.md`, which records eleven plan defects, all authored on the planner side and none originating with an executor. Against them it records two genuine executor errors and one harness error attributable to the supervisor (Table S15). The item-by-item ledger and the diffs that fixed each entry are not released, so a reader cannot check the attribution, cannot see the eleven defects separately, and must take the split on the author’s contemporaneous record. Table S15 collapses eleven distinct defects into one row with one example for that reason, not by choice of presentation. Because the count carries the case study’s central claim and cannot be audited, it is used only in this section and in Methods, where its provenance is stated. It is not used in the Introduction, the Discussion or the Conclusion, which say “repeated human correction” instead. Each of the eleven first presented as a model failure, and each was resolved only by executing the failing step by hand against the live API.

The most expensive defect is the clearest instance. A GFF3 annotation was staged where featureCounts requires a GTF carrying `gene_id`. Six fastp and six STAR jobs ran correctly for approximately 2 h, and then all six counting jobs aborted with `failed to find the gene identifier attribute in the 9th column`. The IWC workflow hard-codes the `exon/gene_id` pair and exposes neither as an input, so the GFF3 could not have worked at any parameterisation. The defect stayed invisible until the last stage of a two-hour workflow, which is what made it expensive rather than merely wrong.

Five defect classes are worth naming for anyone writing plans for executors. The first two are omitted structural keys in an API call, and a step saying “record the output id” without saying which of three available identifiers. The third is a rule forbidding one fallback but not the others, so that a transport error produced 30 min of retries. The last two are matching a file by name rather than by type, and a verification step that states its own expected answer.

The two executor errors are instructive by contrast. One was a transcription slip: the plan supplied a 32-character hexadecimal identifier and the model sent 30, dropping two mid-string, which Galaxy rejected. Literal commands copy reliably because they are lexically structured; opaque identifiers do not, and a two-character elision still looks right. The other error was the fabricated report below.

Plan sufficiency was established during this exercise rather than tested by it, so the corrections are data about plan authoring, not a validated protocol. This case study cannot test specification sufficiency at all: it contains no plan gradient, so nothing in it discriminates a sufficient plan from an insufficient one.

Table S15. Defect ledger for the benchmark-2 case study, attributing each defect to the party that authored it. Source: `revision/galaxy_demo/FACTS.md`, a hand-typed contemporaneous record, not primary data. The full eleven-item ledger with the diff that fixed each entry is not released, so the three counts in this table cannot be independently verified (main text, *Availability*). Either releasing the ledger or dropping the counts would resolve this; releasing it is the right answer and the artifacts do not exist.

Class	n	Attribution	Example
Plan defects	11	Planner (human-supervised)	GFF3 staged where featureCounts requires a GTF with <code>gene_id</code>
Executor errors	2	Model	Two characters dropped from a 32-hex identifier; one fabricated report
Harness error	1	Supervisor	Two runner processes alive in one workspace, truncating and appending to the same files

The seed extension at the discriminating conditions

After the second review cycle we raised the three plan conditions carrying the mechanism argument — v1g, v1.25 and v1.5, Track A, reasoning off — from three to ten seeds per model (seeds 42–51). This seed extension ran across thirteen local executors: the twelve gradient models plus `qwen3.8:27b`. The new cells are in `revision/logs/matrix_jetson_seedext.jsonl` (273 cells) and are pooled with `matrix_jetson_thinkoff.jsonl`. `qwen3.8:27b`’s original three seeds sit in its own log (`matrix_jetson_qwen38_off.jsonl`) and are not pooled, so that model contributes seven runs per condition. One attempted `gpt-oss:20b` v1g cell has no score record and could not be reconstructed, so it is treated as missing, making the denominators 126 for v1g and 127 for v1.25 and v1.5. Treating it as a failure instead gives 31/127 and does not change the qualitative result. Pooled descriptive totals, with Clopper–Pearson exact 95% intervals, are v1g 31/126 (0.246, [0.17, 0.33]), v1.25 63/127 (0.496, [0.41, 0.59]) and v1.5 109/127 (0.858, [0.79, 0.91]).

Repeated seeds from one model are not independent experimental units. The primary comparisons therefore use each model’s success proportion in each condition and apply an exact two-sided sign-flip test to the 13 paired model-level differences. For v1.25 minus v1g the mean within-model difference is 0.253 ($p = 0.027$); for v1.5 minus v1.25 it is 0.354 ($p = 0.0039$). As a run-level sensitivity analysis that preserves pairing by model and seed but not clustering above that level, the matched discordant counts are 39 favouring v1.25 and 7 favouring v1g (exact McNemar $p = 1.8 \times 10^{-6}$), and 52 favouring v1.5 and 6 favouring v1.25 ($p = 3.2 \times 10^{-10}$). The model-level analysis is primary because the scientific claim concerns performance across models.

Table S16. Seed extension at the discriminating conditions: perfect runs ($M3 = 1.0$) per model per plan, Track A, reasoning off, ten seeds per model. `qwen3.8:27b` contributes seven seeds for the reason above; `gpt-oss:20b`’s v1g denominator is 9 because one generation produced no score record.

Model	v1g	v1.25	v1.5
<code>gemma4:26b-a4b</code>	0/10	6/10	7/10
<code>gemma4:31b</code>	7/10	10/10	10/10
<code>glm-4.7-flash</code>	0/10	4/10	10/10
<code>gpt-oss:20b</code>	0/9	7/10	8/10
<code>granite4.1:30b</code>	7/10	5/10	9/10
<code>granite4.1:8b</code>	0/10	0/10	8/10
<code>laguna-xs-2.1</code>	0/10	0/10	10/10
<code>nemotron-3-nano:30b-a3b</code>	2/10	3/10	10/10
<code>nemotron-3-nano:4b</code>	0/10	0/10	7/10
<code>qwen3.5:4b</code>	0/10	3/10	3/10
<code>qwen3.6:27b</code>	10/10	8/10	10/10
<code>qwen3.6:35b-a3b</code>	0/10	10/10	10/10
<code>qwen3.8:27b</code>	5/7	7/7	7/7
Total	31/126	63/127	109/127

The per-model rows show that the pooled v1g figure is not a uniform floor. Four models (`qwen3.6:27b`, `qwen3.8:27b`, `gemma4:31b`, `granite4.1:30b`) handle the registry-style block most of the time, one does occasionally, and eight never do. The pooled contrast is the comparison the manuscript quotes; the per-model heterogeneity is reported, not tested.

The perturbed-plan pass

The design is in the main text (Methods, *The perturbed-plan condition*), and it is a moved-path test: plan v2 is used byte-for-byte verbatim while one input file the plan names has moved. The sandbox holds the same four read pairs, but the reference now sits at `data/ref/GRCh38_chrM/rCRS.fa`, the plan’s literal `data/ref/chrM.fa` does not exist, and the prompt’s DATASET section states the real path. Before any model ran, we validated the sandbox in both directions with two hand-written scripts. A copier, which transcribes the plan’s command lines unchanged, fails at `bwa index`; a binder, which substitutes the stated path — that is, binds the plan to the input actually present — produces the correct call set. Thirteen local executors $\times 3$ seeds = 39 cells, reasoning off, Track A; the log is `revision/logs/matrix_jetson_perturbed.jsonl`, and per-cell artifacts are under `revision/runs_perturbed/`. The control is the unperturbed published v2 condition, 34/36 perfect; no frontier arm was run.

Table S17. Perturbed-plan pass, per model: 6/39 perfect overall, and every model is 3/3 or 0/3. The path column is read from the emitted `run.sh` scripts, not from the scores; all three seeds of each model agree on it. Every failing run exited at the first step with `[bwa_idx_build] fail to open file 'data/ref/chrM.fa'`; several failing models also derived index filenames (e.g. `chrM.fa`) from the stale path.

Model	Perfect runs	Reference path written in the emitted scripts
<code>gemma4:31b</code>	3/3	<code>data/ref/GRCh38_chrM/rCRS.fa</code> (bound to the stated input)
<code>gpt-oss:20b</code>	3/3	<code>data/ref/GRCh38_chrM/rCRS.fa</code> (bound to the stated input)
<code>gemma4:26b-a4b</code>	0/3	<code>data/ref/chrM.fa</code> (copied from the plan)
<code>glm-4.7-flash</code>	0/3	<code>data/ref/chrM.fa</code> (copied from the plan)
<code>granite4.1:30b</code>	0/3	<code>data/ref/chrM.fa</code> (copied from the plan)
<code>granite4.1:8b</code>	0/3	<code>data/ref/chrM.fa</code> (copied from the plan)
<code>laguna-xs-2.1</code>	0/3	<code>data/ref/chrM.fa</code> (copied from the plan)
<code>nemotron-3-nano:30b-a3b</code>	0/3	<code>data/ref/chrM.fa</code> (copied from the plan)
<code>nemotron-3-nano:4b</code>	0/3	<code>data/ref/chrM.fa</code> (copied from the plan)
<code>qwen3.5:4b</code>	0/3	<code>data/ref/chrM.fa</code> (copied from the plan)
<code>qwen3.6:27b</code>	0/3	<code>data/ref/chrM.fa</code> (copied from the plan)
<code>qwen3.6:35b-a3b</code>	0/3	<code>data/ref/chrM.fa</code> (copied from the plan)
<code>qwen3.8:27b</code>	0/3	<code>data/ref/chrM.fa</code> (copied from the plan)

Three cautions bound this table. Each per-model figure is three seeds, so a 3/3 is [0.29, 1.00] and a 0/3 is [0.00, 0.71]; the result is the sharpness of the split across 39 cells, not any per-model rate. The perturbation is a single type — one moved path — and says nothing about renamed samples, added samples, permuted steps or wrong output paths. No frontier executor was run on the perturbed plan, so whether frontier models bind under it is unknown.

The bounded repair arm

The repair arm re-ran the perturbed condition with feedback (main text, Results, *Three attempts separate transcribers that repair from transcribers that do not*). If the generated script exited nonzero, the model saw its own script, the exit code and the last 40 lines of the execution log, and could submit a fix, with at most three attempts allowed. The retry signal was the exit code only; the score was computed afterward and never shown. Thirteen local executors $\times 3$ seeds = 39 cells, reasoning off, Track A; the log is `revision/logs/matrix_jetson_repair.jsonl`, and per-attempt scripts are under `revision/runs_repair/`.

Table S18. The bounded repair arm, per model: attempts used and M3 per seed (seeds 42/43/44). An attempts entry of 1 means the first script succeeded; an entry of 2 means the model repaired after one error report; an entry of 3 means the model used every attempt, and every such seed ended at M3 = 0. Copier seeds rescued: 8 of 33. Class success with repair: 14/39, against 6/39 one-shot and 34/36 unperturbed.

Model	Tier	Attempts (42/43/44)	M3 (42/43/44)
<code>gemma4:31b</code>	bind on attempt 1	1 / 1 / 1	1.0 / 1.0 / 1.0
<code>gpt-oss:20b</code>	bind on attempt 1	1 / 1 / 1	1.0 / 1.0 / 1.0

Model	Tier	Attempts (42/43/44)	M3 (42/43/44)
qwen3.6:27b	repair on attempt 2	2 / 2 / 2	1.0 / 1.0 / 1.0
qwen3.8:27b	repair on attempt 2	2 / 2 / 2	1.0 / 1.0 / 1.0
laguna-xs-2.1	partial repair	2 / 3 / 2	1.0 / 0.0 / 1.0
gemma4:26b-a4b	no repair	3 / 3 / 3	0.0 / 0.0 / 0.0
glm-4.7-flash	no repair	3 / 3 / 3	0.0 / 0.0 / 0.0
granite4.1:30b	no repair	3 / 3 / 3	0.0 / 0.0 / 0.0
granite4.1:8b	no repair	3 / 3 / 3	0.0 / 0.0 / 0.0
nemotron-3-nano:30b-a3b	no repair	3 / 3 / 3	0.0 / 0.0 / 0.0
nemotron-3-nano:4b	no repair	3 / 3 / 3	0.0 / 0.0 / 0.0
qwen3.5:4b	no repair	3 / 3 / 3	0.0 / 0.0 / 0.0
qwen3.6:35b-a3b	no repair	3 / 3 / 3	0.0 / 0.0 / 0.0

The caveats of the perturbed pass carry over: three seeds per model, one perturbation type, no frontier arm. The repair arm adds one of its own, because it retried only on a nonzero exit.

Supplementary Discussion

Candidate workflow classes for a local executor

Table S19. Candidate workflow classes for a local executor.

The compressed six-class table that carried this content in the main text of the first version has been removed: four of its six rows were author opinion with no measurement behind them. A numbered main-text table gave them a standing they had not earned. The boundary they were there to draw is stated in prose in *Scope, study design, and what each benchmark can and cannot show*. The expanded version is retained here for readers who want the reasoning per class, with every qualitative judgement marked. It adds the representative analysis and the reason each class stresses a different part of an executor. Step counts and I/O footprints are measured only for the two classes that were run. For the four classes that were not run, plan complexity and I/O footprint are qualitative estimates by the author with no measurement behind them, and they are marked as such. No numeric step count is offered for them, because no source in the released artifacts gives one.

Workflow class	Representative analysis	Steps in the plan	Plan complexity	I/O footprint	Run here
Alignment and small-variant calling	mtDNA amplicon calling, 4 paired-end MiSeq samples against chrM (16,569 bp)	7 (measured)	Low: a fixed linear chain of shell binaries, no data-dependent parameters	Small: 8 FASTQ files in, per-sample BAM/VCF and one collapsed table out	Yes — benchmark 1
RNA-seq quantification and differential expression	<i>C. auris</i> re-quantification, 6 paired-end runs, 20.5–30.1 M fragments per sample, IWC rnaseq-pe v1.5 (fastp → STAR → featureCounts)	30 (measured)	Moderate: branching workflow, a reference build and an annotation to resolve, 8 parameters to bind	Moderate: 12 FASTQ files, a genome and a GTF in; per-sample count tables out	Quantification yes — benchmark 2 case study. Differential expression not run
Single-cell RNA-seq	Droplet-based expression atlas, filtering through clustering	Estimate (qualitative)	High: filtering thresholds and clustering resolution are scientific judgements with no canonical value to transcribe	Large: raw matrices scale with cell number	No

Workflow class	Representative analysis	Steps in the plan	Plan complexity	I/O footprint	Run here
De novo assembly	Short- or long-read genome assembly with polishing and evaluation	Estimate (qualitative)	High: assembler parameterisation depends on read length, coverage and heterozygosity	Large: memory-bound, long runtimes, large intermediates	No
Metagenomic profiling	Taxonomic and functional profiling against a reference database	Estimate (qualitative)	Moderate: mostly fixed wiring, but database selection and version are decisions	Large: dominated by database residency rather than by sample size	No
ChIP-seq / ATAC-seq peak calling	Treatment-versus-control peak calling with QC	Estimate (qualitative)	Moderate: fixed wiring, but peak-calling thresholds are data-dependent	Moderate: paired treatment and control libraries in, peak files out	No

The table makes one pattern visible: the two classes run here are the two in which the plan can state the answer. The four classes not run include the classes in which it cannot. That statement is repeated in the main text, in *Scope* and in *Scope of the plan-sufficiency result*, because it bounds every claim in this paper.

Production concerns for a shared-cluster deployment

The case study supplies part of the answer to what changes in a production setting, because usegalaxy.org is one. The executor operated under an external scheduler with real queueing, per-tool containerisation and server-side provenance recorded independently of the agent. Its credential was account-scoped, so its blast radius was the account rather than the history, and what narrowed that radius in practice was the MCP server’s bounded verb set, not the credential. Four production concerns remain untested, and for each of them this study has enough behind it to say something more useful than a list (Table S20).

Table S20. Production concerns a shared-cluster deployment must settle, what benchmark 2 supplied, and what this study’s own failures recommend. No row is a tested result.

Concern	What benchmark 2 supplied	What remains untested	Recommended mitigation
Executor process isolation	Nothing: the local arm ran unsandboxed on the board that hosted the model, with no container, cgroup or namespace	Multi-user isolation of the executor process; whether generated code can reach another user’s files	A container or per-user cgroup around the executor is a prerequisite, not a hardening step (<i>Execution environment</i> , above). The one out-of-sandbox action observed — an unbounded <code>rm -f /tmp/*.txt</code> — is exactly what a shared <code>/tmp</code> makes dangerous
Credential scoping	An account-scoped Galaxy API key, whose blast radius was the whole account; what narrowed it was the MCP server’s bounded verb set, not the credential	Whether per-run credentials can be issued and revoked at the scale of a queue	A per-run service account, a dedicated history with a scoped key, and an MCP server exposing only the invoke, poll and read verbs. This is stated in Methods and belongs here, where a deployment is being planned
Queue-time behaviour	A multi-hour polling loop against a real scheduler, which required a fresh-session flag and a hard TERM timeout to survive	What happens when the polling loop outlives the session that started it, or when the queue delays a job past any session budget	Durable state plus an explicit session-restart contract; treat the poller as a supervised process with its own timeout rather than as a turn in a conversation

Concern	What benchmark 2 supplied	What remains untested	Recommended mitigation
Unattended failure notification	The concrete failure: six counting jobs aborted approximately two hours into an unattended run, with no notification path, and were noticed by a human	Any paging or alerting path at all; none existed here	A failure channel outside the agent, since the agent that fails is the one that would have to report — which is the same lesson as the unexecuted reporting step

Agent harnesses and bioinformatics agents

The execution side of a planner-executor system is supplied in practice by an *agent harness* — software that presents tools to a model, runs the resulting tool calls, feeds the outputs back, and decides when to stop. Several open-source harnesses were in wide use by mid-2026 — Claude Code, OpenCode, Aider, OpenHands, Goose, Cline and Continue among them — and no comparison among them was run here. The first version of this manuscript carried a capability table for these harnesses. It is removed because every row answered yes to every capability column, so no reader could check any cell against a pinned release. Benchmark 2 used `galaxyproject/loom` for two reasons that are properties of the deployment rather than judgements about the alternatives. It runs headless with no terminal attached, and it keeps a file-backed `notebook.md` that survives session boundaries, which the multi-hour Galaxy polling loop requires. Nothing in the results is known to be harness-independent, and the failure modes reported for benchmark 2 — a stale durable note, an unexecuted final step — are partly properties of that harness.

Benchmark 1’s harness is not an agent harness at all: it issues one prompt, receives one script, and executes it externally. It has no loop, shows the model no output, and gives it no opportunity to repair. We chose that design deliberately, because it isolates plan detail as the only variable, and it is also precisely the limitation three reviewers of the first version of this manuscript identified. The resulting system should not be called agentic, and benchmark 2 was added to supply the missing loop.

Within bioinformatics, the closest work evaluates open-ended tool-using research agents against task suites. Biomni couples a large tool and database inventory to a frontier model and scores the system across a wide range of biomedical tasks, including ones requiring literature retrieval and experimental design (Huang et al. 2025). BixBench grades agents on 53 Dockerised computational-biology capsules with 296 curated open-answer questions (Mitchener et al. 2025). BioMaster wraps a plan/task/debug/check loop around retrieval-augmented generation for RNA-seq, ChIP-seq, scRNA-seq and Hi-C (Su et al. 2025), and BioAgents fine-tunes small open-weight models for local execution (Mehandru et al. 2025). We did not run any of these systems on this study’s task, and no external anchor was obtained. None of the numbers reported here can be placed on a published scale, so a reader cannot tell from this paper whether 34 of 36 perfect runs at the detailed plan is good by any external standard. BixBench ships 53 public Dockerised capsules precisely so that new executors can be placed on a common scale. Running this study’s executors on a subset of them is a bounded experiment on public artifacts that would convert this declination into an anchor, and it was not run. This is the sharpest reproducibility gap in the paper’s positioning. The first version of this manuscript carried a table contrasting scoring conventions across benchmarks. Every row of it explained why the numbers are not comparable, which is an apology rather than an anchor, and it too is removed.

Two external comparators were available for benchmark 2, and both are used with their protocols stated. A human executed the same Galaxy workflow manually on the same server. And the Galaxy Project published a run on 9 June 2026 in which eight frontier models acted as *both* planner and executor on the same RNA-seq data (Nekrutenko 2026): cost was \$2.82–\$131.83 per run, and six of eight models reproduced the original finding. That post is not matched to this study: it is by the present author, its models plan as well as execute, its plans and scoring rubric are not published, and it reports no replicates. It bounds the API cost of a comparable task but is not a frontier control for anything measured here.

Verbatim prompts and plan files

Prompt structure for the planner and the executor

Nine files define everything a model saw in benchmark 1, and all of them are reproduced verbatim in *The files in full* below; we first summarise their structure here, by category. Three planner meta-prompts — `plan/PLANNER_PROMPT.md`, `plan/PLANNER_PROMPT_v2.md`, and `plan/PLANNER_PROMPT_v2_defensive.md` — instructed a planning model to write the plan a later executor would receive. One executor system prompt, `prompts/system.txt`, fixed the rules under which the script-writing model worked. Three executor user templates carried the task itself. `prompts/track_a_user.tmpl` serves Track A; `prompts/track_b_user.tmpl` serves Track B; and the Track-B-with-order variant `prompts/track_b_with_order_user.tmpl` constitutes the v0.5 condition. Plan files v1 and v2 complete the set.

Methods (*Plan conditions*) categorises the planner meta-prompts on five axes: role and audience framing; whether bash code is permitted in the plan; whether literal flag values are required; the step-count limit; and whether validation and retry prose is requested. `PLANNER_PROMPT.md` forbids code, caps the plan at ten steps and 400 words, requires flags to be named but not shown, and requests no validation prose. `PLANNER_PROMPT_v2.md` requires a fenced literal one-liner per step with every flag and value, an idempotency guard, and a gotchas block; it sets no step limit and tells the planner that the implementer will copy-paste its invocations. `PLANNER_PROMPT_v2_defensive.md` adds the defensive block — the validation, retry, and failure-logging requirements reproduced in full below. The transcription argument holds that a detailed plan helps because the executor copies its literal strings rather than recalling tool knowledge. Readers checking that argument should note one fact: `PLANNER_PROMPT.md`, the meta-prompt behind the *lean* condition, itself contains the literal read-group string and the `lofreq call-parallel` subcommand name.

The system prompt fixes the role (“an expert bioinformatician”) and the output contract: one self-contained bash script, no prose, no markdown fences. It then imposes seven hard constraints on the emitted script. The script must open with the shebang line and `set -euo pipefail`, assume a working directory containing `data/raw/` and `data/ref/`, and write all output under `results/`. It must also be idempotent on re-run, use no absolute paths outside the working directory, call only the tools in the injected inventory with no package managers, and finish within a 600 s wall-clock limit at four threads. Three of these are enforced by the harness — the wall-clock kill, the tool inventory on `PATH`, and the working directory — while the remaining four are requests the model can violate. We measure the rate at which it did in the Results (*What the emitted scripts actually did*).

The user template has five slots in a fixed order: the task specification, the dataset manifest, the tool inventory, the plan slot, and the output-format reminder. The task specification covers per-sample variant calling on four paired-end samples and gives the exact list of required output paths together with the exact column header of the collapsed table. The dataset manifest names every input file, and we inject the tool inventory at run time from `setup/verify_env.sh`, so the prompt cannot drift from the environment. Track A and Track B differ only in the plan slot: Track A substitutes the plan file, while Track B substitutes the sentence “None provided. Design and implement the workflow yourself, using only the tools above.” Because the Track B template contains no plan placeholder, every Track B run is a no-plan run regardless of which plan file was nominally passed to the harness. That rule must be honoured in any re-analysis of the released data. The dataset manifest also carries this study’s own subject labels, which is why we note in Methods that the frontier executor arms cross the governance boundary the planner role respects.

Safety and validation checks are carried by exactly one plan condition. `v2_defensive` adds a `try()` helper, per-step validation, retry-once semantics, a `failures.log`, and an exit-code policy; the other seven plan conditions carry none of these. That difference is a measured experimental condition, not an oversight — it is the comparison behind the claim that a defensive plan changes error handling. The Results narrow that claim, because the size and sign of the change are model-dependent.

The Galaxy plan was structured differently because the executor there is a loop rather than a single generation. The plan is a 50-line `PLAN.md` index plus seven step files, so the model loads only the step it is currently executing. The seventh file is the verification step, and we rewrote it mid-study to state no expected values, to require a dataset identifier beside every reported number, and to sanction `UNAVAILABLE` as an answer. The

original stated its own expected values (“~5,594 genes, ~92% assigned”) and would therefore let an executor that never opened a dataset produce a flawless verification by recitation. Neither version of that plan is in the released repository, so neither can be reproduced here. The general point stands independently of the lost artifact: any agent evaluation placing the expected answer in the task description cannot distinguish execution from recitation.

The files in full

The first version of this manuscript stated that these files were reproduced below the tables and did not reproduce them: the pointer existed, the material did not. Reviewers asked specifically that pointing at a repository is not sufficient and that the material be lifted into the paper. They were right. We therefore give in full below the three planner meta-prompts, the executor system prompt, and the three executor user templates. We also reproduce the two plan files every gradient argument turns on: v1 and v2, the ends of the ladder of plan detail on which the paper’s central comparisons rest. We omit plan files v1.25, v1.5, v1g and v2_defensive for length; they are in `plan/` in the released repository.

`plan/PLANNER_PROMPT.md`

The planner meta-prompt that produced plan v1; plans v1.25 and v1.5 were then derived by editing v1.

You are designing a per-step implementation plan for a junior bioinformatician. The implementer will read

Audience: a competent shell programmer who knows generic bioinformatics tools but not the specific best

Workflow goal

Per-sample variant calling on 4 paired-end MiSeq amplicon samples mapped to the human mitochondrial ref

```
results/{sample}.bam
results/{sample}.bam.bai
results/{sample}.vcf.gz
results/{sample}.vcf.gz.tbi
results/collapsed.tsv          # columns: sample, chrom, pos, ref, alt, af
```

Inputs already on disk

- data/raw/{sample}_1.fq.gz, {sample}_2.fq.gz for sample in {M117-bl, M117-ch, M117C1-bl, M117C1-ch}
- data/ref/chrM.fa (decompressed but not indexed)

Tools available (only these; pinned in a conda env)

{TOOL_INVENTORY}

Your task

Write a numbered, ≤ 10 -step plan covering:

1. Reference preparation (which indexes are needed, which tools, and in what order).
2. Per-sample alignment (specify the aligner and threading). The read-group string passed to ``bwa mem -l``
 - The separators between key and value are colons (``:``), not equals signs (``=``).
 - The separators between fields are the two literal characters ``\`` and ``t`` (a backslash followed by ``t``).
3. BAM post-processing (sort, index; whether duplicate marking is needed for amplicon data - note that a
4. Variant calling (specify caller, threading, parallelization choice, and the output format).
5. VCF post-processing (compression and indexing; what extension and indexer).
6. Collapse step (specify exactly how to derive the collapsed.tsv: which fields, which separator, header
7. Idempotency expectations (re-running on a populated results/ dir must exit 0 without redoing finished

Constraints to mention to the implementer:

- THREADS=4 throughout.
- Use the "lofreq call-parallel" subcommand for variant calling.

- Use bgzip + tabix for VCF compression and indexing (not "bcftools view -O z" - be explicit).
- Do not perform duplicate marking; this is amplicon data.
- Use bcftools query for the collapse step; the AF tag is in INFO.

Output format: a single Markdown document with a numbered list and short descriptive bullets. No code blocks.

plan/PLANNER_PROMPT_v2.md

The planner meta-prompt that produced plan v2, the detailed condition of the plan gradient.

You are designing an EXTREMELY DETAILED, ZERO-AMBIGUITY implementation recipe for a junior bash programmer.

You will not write the bash code yourself - that's the implementer's job. But your plan must specify, for each step:

- exact tool name and subcommand
- every flag, with its exact value
- every input file path, with the exact form expected
- every output file path
- the exact relative ordering and what stdin/stdout pipes to where

Audience constraint: assume the implementer will copy-paste your invocations into a shell script. If you

Workflow goal

Per-sample variant calling on 4 paired-end MiSeq amplicon samples mapped to the human mitochondrial reference genome.

```
results/{sample}.bam
results/{sample}.bam.bai
results/{sample}.vcf.gz
results/{sample}.vcf.gz.tbi
results/collapsed.tsv          # columns: sample, chrom, pos, ref, alt, af
```

Inputs already on disk

- data/raw/{sample}_1.fq.gz, {sample}_2.fq.gz for sample in {M117-bl, M117-ch, M117C1-bl, M117C1-ch}
- data/ref/chrM.fa (decompressed, but NOT yet indexed)

Tools available (only these; pinned in a conda env)

{TOOL_INVENTORY}

Required content of your plan

For each numbered step, give:

1. The exact command line (or pipeline) the implementer must run. Use `{sample}` as a placeholder and t
2. What that step's output file is (exact path).
3. An idempotency guard: the exact `[[ -f ... ]] || ...` test that would skip this step.
4. Any tool-specific gotchas (e.g. "bwa rejects real tab characters in -R; the literal text must be back

Cover every step end-to-end:

- a. Reference preparation: `bwa index data/ref/chrM.fa` and `samtools faidx data/ref/chrM.fa`. Specify w
- b. Per-sample alignment: `bwa mem -t 4 -R "@RG\tID:{sample}\tSM:{sample}\tLB:{sample}\tPL:ILLUMINA" data
- c. BAM indexing: `samtools index -@ 4 results/{sample}.bam`. Output: `results/{sample}.bam.bai`. Do NOT
- d. Variant calling: the EXACT command is `lofreq call-parallel --pp-threads 4 -f data/ref/chrM.fa -o re
- e. VCF compression and indexing: `bgzip -f results/{sample}.vcf` (this removes `results/{sample}.vcf` an
- f. Collapse step: write a single header line `printf 'sample\tchrom\tpos\tref\talt\taf\n' > results/col

Required at the top of the plan:

- A ``set -euo pipefail`` directive expectation.
- A ``THREADS=4`` constant expectation.
- A ``SAMPLES=("M117-bl" "M117-ch" "M117C1-bl" "M117C1-ch")`` array expectation.
- A ``mkdir -p results`` step.

Required at the bottom:

- An idempotency note: every step must guard against re-doing work, and a second invocation on a populated directory must be a no-op.
- A re-build note: rebuild `results/collapsed.tsv` from scratch on each run (cheap; avoids stale rows).

Style: numbered Markdown list with code-fenced invocations. ≤ 700 words. Do not include the bash boilerplate.

plan/PLANNER_PROMPT_v2_defensive.md

The planner meta-prompt that produced plan v2_defensive; it is the v2 prompt plus the defensive-execution block.

You are designing an EXTREMELY DETAILED, ZERO-AMBIGUITY implementation recipe for a junior bash programmer.

You will not write the bash code yourself - that's the implementer's job. But your plan must specify, for every step:

- exact tool name and subcommand
- every flag, with its exact value
- every input file path, with the exact form expected
- every output file path
- the exact relative ordering and what stdin/stdout pipes to where

****Critical defensive-execution requirement.**** Some tool invocations will fail at runtime - transient I/O errors.

- ****Output validation after every per-sample step.**** After every step that writes a file (alignment, index, etc.) must be validated.
- ****Retry-once on transient failure.**** Specify that any tool invocation that exits non-zero MUST be retried once.
- ****Per-sample isolation.**** A failure of one sample MUST NOT abort the whole workflow. Other samples must continue.
- ****Failure logging.**** The script must record, in a structured way, which sample $\times$ step combinations fail.
- ****Final summary line on stderr.**** The very last line on stderr must be a single human-readable summary.
- ****Exit code policy.**** The script exits 0 if AT LEAST ONE sample completes the full pipeline through the script.
- ****`set -euo pipefail` is still required****, but every command that may legitimately fail must be wrapped in `|| true`.

Audience constraint: assume the implementer will copy-paste your invocations into a shell script. If you need to, you can use `<code>` for code blocks.

Workflow goal

Per-sample variant calling on 4 paired-end MiSeq amplicon samples mapped to the human mitochondrial reference.

```
results/{sample}.bam
results/{sample}.bam.bai
results/{sample}.vcf.gz
results/{sample}.vcf.gz.tbi
results/collapsed.tsv      # columns: sample, chrom, pos, ref, alt, af
results/failures.log       # NEW: tab-separated sample\tstep\treason rows for any per-sample failures
```

Inputs already on disk

- data/raw/{sample}_1.fq.gz, {sample}_2.fq.gz for sample in {M117-bl, M117-ch, M117C1-bl, M117C1-ch}
- data/ref/chrM.fa (decompressed, but NOT yet indexed)

Tools available (only these; pinned in a conda env)

{TOOL_INVENTORY}

Required content of your plan

For each numbered step, give:

1. The exact command line (or pipeline) the implementer must run.
2. What that step's output file is (exact path).
3. An idempotency guard: the exact ``[[ -f ... ]] || ...`` test that would skip this step.
4. **An output-validation command** (after the step runs): the exact one-liner that returns 0 iff the output is valid.
5. **A retry-once-then-skip pattern**: tell the implementer to retry the step exactly once on failure (e.g. ``[[ $RC != 0 ]] || sleep 10; $CMD``).
6. Any tool-specific gotchas (e.g. "bwa rejects real tab characters in -R; the literal text must be backslash-tab").

Cover every step end-to-end:

- a. Reference preparation: ``bwa index data/ref/chrM.fa`` and ``samtools faidx data/ref/chrM.fa``. These are the only files that need to be indexed.
- b. Per-sample alignment: ``bwa mem -t 4 -R "@RG\tID:{sample}\tSM:{sample}\tLB:{sample}\tPL:ILLUMINA" data/{sample}.fastq.gz data/ref/chrM.fa``.
- c. BAM indexing: ``samtools index -@ 4 results/{sample}.bam``. Output: ``results/{sample}.bam.bai``. Validation: ``samtools view -h results/{sample}.bam | wc -l``.
- d. Variant calling: the EXACT command is ``lofreq call-parallel --pp-threads 4 -f data/ref/chrM.fa -o results/{sample}.vcf``.
- e. VCF compression and indexing: ``bgzip -f results/{sample}.vcf`` then ``tabix -p vcf results/{sample}.vcf.gz``.
- f. Collapse step: only over the **surviving** samples (those that produced a valid ``vcf.gz.tbi``). Header: ``samtools view -h results/{sample}.bam | wc -l``.

Required at the top of the plan:

- A ``set -euo pipefail`` directive expectation.
- A ``THREADS=4`` constant expectation.
- A ``SAMPLES=("M117-bl" "M117-ch" "M117C1-bl" "M117C1-ch")`` array expectation.
- A ``mkdir -p results`` step.
- A defensive helper or pattern (e.g. ``try() { ... }``) the implementer can reuse. Specify its signature.
- An empty-on-creation ``results/failures.log`` (header optional but consistent).

Required at the bottom:

- The exact final-summary stderr line format (see above).
- The exact exit-code policy (0 if any sample fully succeeded, 1 otherwise).
- An idempotency note: every step must guard against re-doing work; a second invocation on a populated results/ directory must exit 0 without redoing work or erroring.

Style: numbered Markdown list with code-fenced invocations. ≤ 900 words. Do not include the bash boilerplate.

prompts/system.txt

The executor system prompt; the same text was used, unchanged, in every benchmark-1 cell.

You are an expert bioinformatician. Produce ONE self-contained bash script that performs the requested work.

Hard constraints on the script:

- First line must be: ``#!/usr/bin/env bash``
- Second line must be: ``set -euo pipefail``
- Assume the current working directory contains data/raw/ (with the listed FASTQ files) and data/ref/ (with the listed reference files).
- Write all outputs under results/ (relative to cwd).
- Be idempotent: rerunning on a populated results/ directory must exit 0 without redoing work or erroring.
- Do not hardcode any path outside the current working directory. No /home/anton, no /tmp, no absolute paths.
- Use only tools listed in TOOL_INVENTORY. Do not invoke conda, pip, apt, curl, wget, or any package manager.
- Total wall-clock must be under 600 seconds at THREADS=4.
- Do not write to stderr/stdout beyond what tools naturally emit; do not use ``echo`` for user-facing progress.

prompts/track_a_user.tmpl

The executor user template for Track A; a plan file is substituted for the {PLAN} placeholder at run time.

TASK

====

Per-sample variant calling on 4 paired-end MiSeq amplicon samples mapped to the human mitochondrial ref

```
results/{sample}.bam
results/{sample}.bam.bai
results/{sample}.vcf.gz
results/{sample}.vcf.gz.tbi
results/collapsed.tsv
```

The collapsed.tsv must have columns (tab-separated, with header):

```
sample  chrom  pos  ref  alt  af
```

DATASET (already present under data/raw/ and data/ref/)

=====

Samples:

```
M117-bl    (paired)
M117-ch    (paired)
M117C1-bl  (paired)
M117C1-ch  (paired)
```

Files in data/raw/:

```
M117-bl_1.fq.gz  M117-bl_2.fq.gz
M117-ch_1.fq.gz  M117-ch_2.fq.gz
M117C1-bl_1.fq.gz M117C1-bl_2.fq.gz
M117C1-ch_1.fq.gz M117C1-ch_2.fq.gz
```

File in data/ref/:

```
chrM.fa          (decompressed; not yet indexed)
```

TOOL_INVENTORY (only these are on PATH; pinned versions in conda env 'bench')

=====

{TOOL_INVENTORY}

PLAN (authoritative - follow the ordering and tool choices)

=====

{PLAN}

OUTPUT FORMAT

=====

A single bash script. No markdown fences, no commentary. Just the script.

prompts/track_b_user.tmpl

The executor user template for Track B, which contains no plan slot at all.

TASK

====

Per-sample variant calling on 4 paired-end MiSeq amplicon samples mapped to the human mitochondrial ref

```
results/{sample}.bam
results/{sample}.bam.bai
results/{sample}.vcf.gz
results/{sample}.vcf.gz.tbi
results/collapsed.tsv
```

The collapsed.tsv must have columns (tab-separated, with header):
sample chrom pos ref alt af

DATASET (already present under data/raw/ and data/ref/)

=====

Samples:

M117-bl (paired)
M117-ch (paired)
M117C1-bl (paired)
M117C1-ch (paired)

Files in data/raw/:

M117-bl_1.fq.gz M117-bl_2.fq.gz
M117-ch_1.fq.gz M117-ch_2.fq.gz
M117C1-bl_1.fq.gz M117C1-bl_2.fq.gz
M117C1-ch_1.fq.gz M117C1-ch_2.fq.gz

File in data/ref/:

chrM.fa (decompressed; not yet indexed)

TOOL_INVENTORY (only these are on PATH; pinned versions in conda env 'bench')

=====

{TOOL_INVENTORY}

PLAN

=====

None provided. Design and implement the workflow yourself, using only the tools above.

OUTPUT FORMAT

=====

A single bash script. No markdown fences, no commentary. Just the script.

prompts/track_b_with_order_user.tmpl

The executor user template for the v0.5 condition: Track B plus one line naming the tool order.

TASK

=====

Per-sample variant calling on 4 paired-end MiSeq amplicon samples mapped to the human mitochondrial ref

results/{sample}.bam
results/{sample}.bam.bai
results/{sample}.vcf.gz
results/{sample}.vcf.gz.tbi
results/collapsed.tsv

The collapsed.tsv must have columns (tab-separated, with header):

sample chrom pos ref alt af

DATASET (already present under data/raw/ and data/ref/)

=====

Samples:

M117-bl (paired)
M117-ch (paired)

M117C1-bl (paired)
M117C1-ch (paired)

Files in data/raw/:

M117-bl_1.fq.gz M117-bl_2.fq.gz
M117-ch_1.fq.gz M117-ch_2.fq.gz
M117C1-bl_1.fq.gz M117C1-bl_2.fq.gz
M117C1-ch_1.fq.gz M117C1-ch_2.fq.gz

File in data/ref/:

chrM.fa (decompressed; not yet indexed)

TOOL_INVENTORY (only these are on PATH; pinned versions in conda env 'bench')

=====

```
{TOOL_INVENTORY}
```

PLAN

=====

No detailed plan provided. Recommended tool sequence: bwa → samtools → lofreq → bcftools → awk (for

OUTPUT FORMAT

=====

A single bash script. No markdown fences, no commentary. Just the script.

plan/PLAN_v1.md

Plan v1, the lean condition: tools and ordering are named, but no literal command lines are shown.

Per-sample mtDNA amplicon variant-calling plan

1. ****Set globals and prepare results directory****

- Define `THREADS=4` and the sample list: `M117-bl M117-ch M117C1-bl M117C1-ch`.
- Create `results/` if missing. Use `set -euo pipefail`.
- Treat every output step as idempotent: guard each artifact with an existence check (e.g. skip if `:`

2. ****Reference indexing (once, in `data/ref/`)****

- `samtools faidx data/ref/chrM.fa` → produces `chrM.fa.fai`.
- `bwa index data/ref/chrM.fa` → produces the `.amb .ann .bwt .pac .sa` set.
- Skip both if the index files already exist.

3. ****Per-sample alignment with `bwa mem`****

- Use `bwa mem -t 4` with the paired FASTQs `data/raw/{sample}\_1.fq.gz` and `data/raw/{sample}\_2.fq.gz`.
- Pass the read group via `-R` as a single double-quoted argument containing literal backslash-t betw
- exact form: `-R "@RG\tID:{sample}\tSM:{sample}\tLB:{sample}\tPL:ILLUMINA"`
- The `\t` must remain the two characters backslash and `t` - bwa parses them itself. Do NOT use ` `
- Separators between key and value are colons `:`, not `=`.

4. ****SAM → sorted BAM****

- Pipe `bwa mem` stdout into `samtools sort -@ 4 -o results/{sample}.bam`.
- Do NOT run `markdup` or `rmdup`: this is amplicon data where PCR duplicates are expected and biolog

5. ****BAM indexing****

- `samtools index -@ 4 results/{sample}.bam` → `results/{sample}.bam.bai`.

6. ****Variant calling with `lofreq call-parallel`****

- Use the ``call-parallel`` subcommand (not plain ``lofreq call``) with ``--pp-threads 4``.
 - Reference: ``data/ref/chrM.fa``. Input: ``results/{sample}.bam``.
 - Write uncompressed VCF to a temporary path (e.g. ``results/{sample}.vcf``); lofreq emits plain VCF.
7. ****VCF compression and indexing****
- Compress with ``bgzip`` (not ``bcftools view -O z``) producing ``results/{sample}.vcf.gz``.
 - Index with ``tabix -p vcf results/{sample}.vcf.gz`` → ``results/{sample}.vcf.gz.tbi``.
 - Remove the intermediate uncompressed ``.vcf``.
8. ****Collapse step → `results/collapsed.tsv`****
- For each sample, run ``bcftools query -f '{sample}\t%CHROM\t%POS\t%REF\t%ALT\t%INFO/AF\n' results/{sample}.vcf.gz``.
 - Concatenate all four samples' output.
 - Prepend a single header line ``sample\tchrom\tpos\tref\talt\taf`` (tab-separated).
 - Output is tab-separated, one variant per line, header on, written to ``results/collapsed.tsv``. Rebuild index.
9. ****Idempotency check****
- Final pass: re-running the script on a fully populated ``results/`` exits 0, performs no work, and logs nothing.

plan/PLAN.md

Plan v2, the detailed condition: every command line appears as a fenced literal the executor can copy.

Implementation Plan: Per-sample mtDNA Variant Calling

Boilerplate (top of `run.sh`)

- First line after shebang: ``set -euo pipefail``.
- Constants: ``THREADS=4`` and ``SAMPLES=("M117-bl" "M117-ch" "M117C1-bl" "M117C1-ch")``.
- Create output dir: ``mkdir -p results``.
- All per-sample steps must be wrapped in ``for sample in "${SAMPLES[@]}"; do ... done``.

1. Reference indexing - BWA

...

`bwa index data/ref/chrM.fa`

...

- Outputs (5 sibling files): ``data/ref/chrM.fa.amb``, ``.ann``, ``.bwt``, ``.pac``, ``.sa``.
- Idempotency guard: ``[[ -f data/ref/chrM.fa.bwt ]] || bwa index data/ref/chrM.fa``
- Gotcha: ``bwa index`` writes outputs next to the input; the dir must be writable. No flags needed for a

2. Reference indexing - samtools faidx

...

`samtools faidx data/ref/chrM.fa`

...

- Output: ``data/ref/chrM.fa.fai``.
- Guard: ``[[ -f data/ref/chrM.fa.fai ]] || samtools faidx data/ref/chrM.fa``

3. Per-sample alignment + sort (one pipeline)

...

`bwa mem -t 4 -R "@RG\tID:{sample}\tSM:{sample}\tLB:{sample}\tPL:ILLUMINA" data/ref/chrM.fa data/raw/{sample}.fastq.gz`

...

- Output: `results/{sample}.bam`.
- Guard: `[[-f results/{sample}.bam]] || { bwa mem ... | samtools sort ... ; }` - wrap the whole pipeline
- RG string gotchas (CRITICAL):
 - Use colons (`ID:`, `SM:`, `LB:`, `PL:`) - never `=`.
 - Use the **literal two characters** `\\t` (backslash + t) inside the double-quoted string. Do NOT use `t`.
 - The whole `-R` value must be a single double-quoted argument.
- `samtools sort` trailing `-` reads from stdin.

4. BAM index

...

```
samtools index -@ 4 results/{sample}.bam
```

...

- Output: `results/{sample}.bam.bai`.
- Guard: `[[-f results/{sample}.bam.bai]] || samtools index -@ 4 results/{sample}.bam`
- Do NOT run `markdup` - this is amplicon data; PCR duplicates are expected and informative.

5. Variant calling - LoFreq

...

```
lofreq call-parallel --pp-threads 4 -f data/ref/chrM.fa -o results/{sample}.vcf results/{sample}.bam
```

...

- Output: `results/{sample}.vcf` (uncompressed).
- Guard: `[[-f results/{sample}.vcf || -f results/{sample}.vcf.gz]] || lofreq call-parallel --pp-threads 4 -f data/ref/chrM.fa -o results/{sample}.vcf results/{sample}.bam` (Check both because step 6 will delete the `.vcf` and leave `.vcf.gz`.)
- Gotchas: BAM is positional, NOT behind `-b`/`-i`. The flag is `--pp-threads`, not `-t` or `--threads`.

6. VCF compression + tabix index

...

```
bgzip -f results/{sample}.vcf
```

...

...

```
tabix -p vcf results/{sample}.vcf.gz
```

...

- Outputs: `results/{sample}.vcf.gz` and `results/{sample}.vcf.gz.tbi`.
- Combined guard: `[[-f results/{sample}.vcf.gz.tbi]] || { bgzip -f results/{sample}.vcf && tabix -p vcf results/{sample}.vcf.gz ; }
- Gotchas: `bgzip` operates **in place** - it deletes `results/{sample}.vcf` after writing `results/{sample}.vcf.gz`.

7. Collapsed TSV (rebuild every run)

Do NOT guard this step - always overwrite, since per-sample VCFs may have changed.

Header (overwrite):

...

```
printf 'sample\tchrom\tpos\tref\talt\taf\n' > results/collapsed.tsv
```

...

Per sample, append:

```
bcftools query -f '%CHROM\t%POS\t%REF\t%ALT\t%INFO/AF\n' results/{sample}.vcf.gz | awk -v s={sample} 'B'
```

- Gotchas:

- The format string uses `%INFO/AF`, not `%AF` - bcftools requires the `INFO/` prefix for INFO fields
- The `\t` and `\n` inside `-f '... '` are **bcftools** format codes, parsed by bcftools itself; keep
- awk's `OFS="\t"` is required so `print s,\$0` joins with a tab (`\$0` already contains the tabbed bcf
- Use `>` for the header line, `>>>` for every per-sample append.

Idempotency summary

- Steps 1-6 each have a `[ -f <sentinel> ] ||` guard on their final output. A second invocation on a p
- Step 7 is intentionally rebuilt from scratch on every run (header `>`, then append per sample). This

Supplementary figures

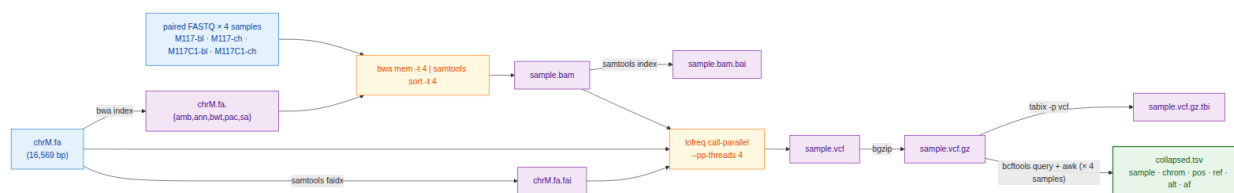


Figure 1: mtDNA workflow DAG

Figure S1. The benchmark-1 reference workflow. **bwa index** and **samtools faidx** prepare the 16,569-bp chrM reference; each of the four samples is aligned with **bwa mem** piped to **samtools sort**, indexed, called with **lofreq call-parallel**, then compressed and indexed with **bgzip** and **tabix**. Only the terminal **bcftools query** plus **awk** fan-in that builds **collapsed.tsv** couples the samples to one another.

Figure S2. Plan gradient with reasoning enabled: same axes, hardware and scoring as main-text Figure 2. Both Granite models are absent because ollama rejects the **think** parameter for them outright (HTTP 400); truncated generations are scored 0 and retained, per the single exclusion rule in Methods.

Figure S3. Effect of enabling reasoning, by model and plan condition: mean M3 with reasoning on minus mean M3 with reasoning off, over the ten models runnable in both arms. Positive (red) means reasoning helped; negative (blue) means it hurt. The negative band at the right-hand edge is the v2 regression.

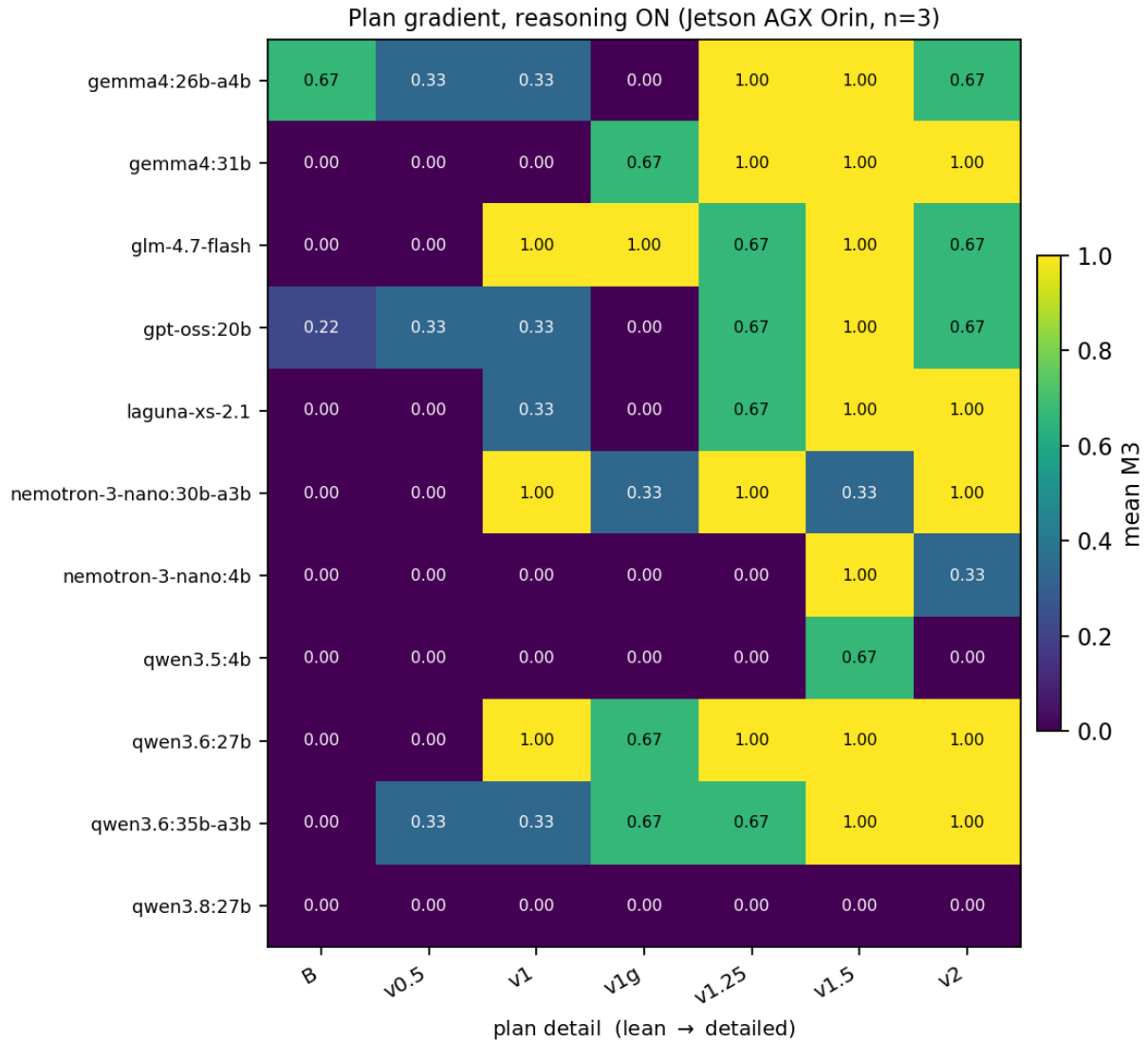


Figure 2: Plan gradient with reasoning enabled

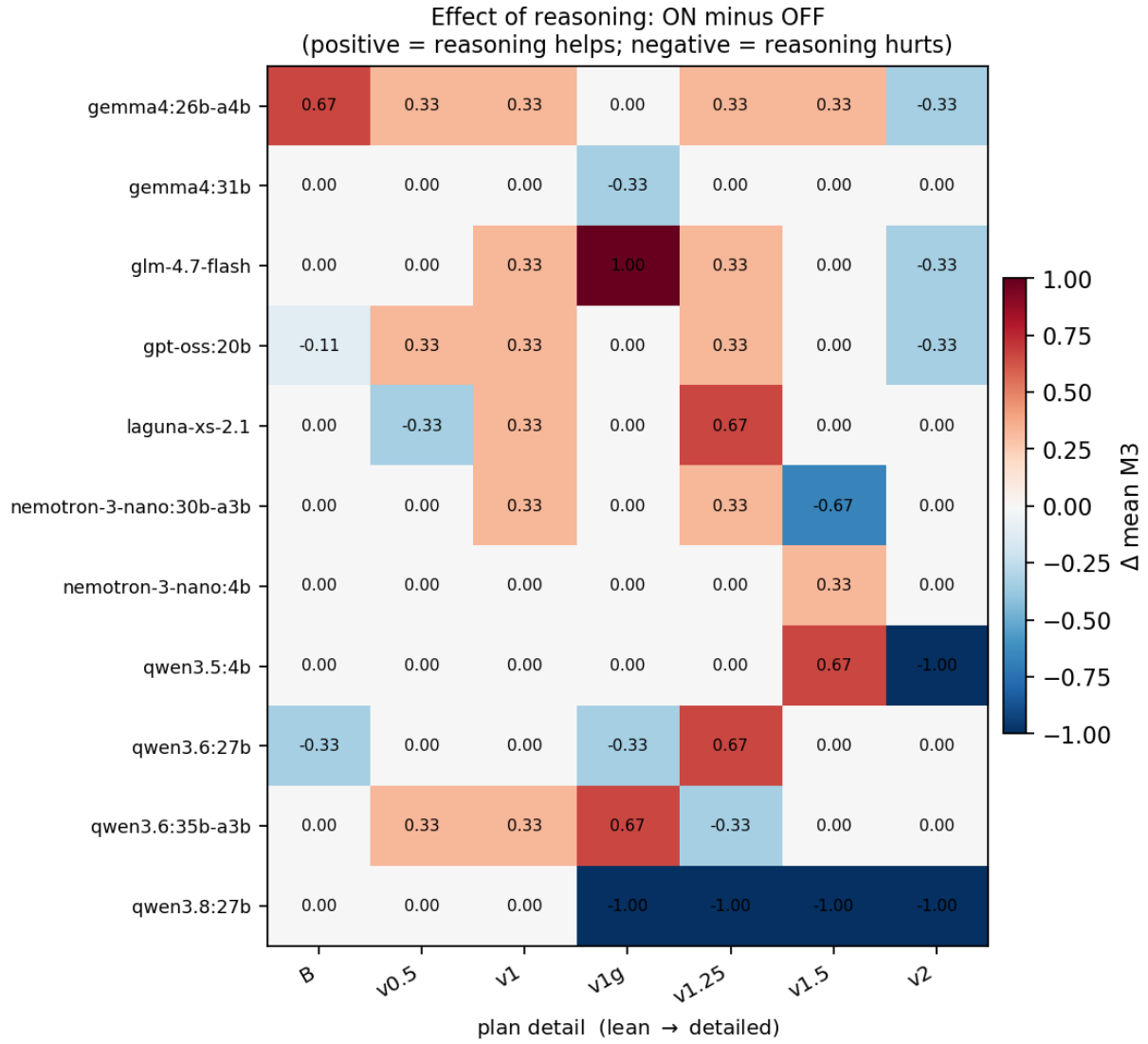


Figure 3: Effect of enabling reasoning